



MORPHOIA

MORPHOIA ENGINE

Note d'opportunité, architecture de référence et plan de réalisation

Une plateforme ouverte d'interopérabilité entre CAO, maillages, nuages de points, imagerie médicale, simulation, intelligence artificielle et calcul haute performance.

VERSION 0.2 • 6 AOÛT 2026

Auteur et porteur du projet : Dr Olivier Ami

Statut : architecture corrigée • profil d'intégration SALOME 9.16

Décision en une page

Morphoia ne doit pas réécrire un moteur 3D monolithique. Il doit devenir la couche ouverte qui rend compatibles, traçables et exécutables des représentations 3D hétérogènes sur postes de travail, GPU et infrastructures EuroHPC.

Décision	Arbitrage retenu	Effet
Positionnement	Infrastructure d'interopérabilité 3D scientifique	Évite une concurrence frontale inutile avec SALOME, ParaView, Slicer ou Blender.
Cœur	C++20 + ABI C versionnée + Python	Performance native, compatibilité binaire et adoption recherche.
Données	IR fédérée, pas de format universel	Préserve B-Rep, mesh, points, voxels, champs et tenseurs sans perte implicite.
CAO	OCCT/XDE + profil STEP/AP242 Morphoia	Réemploi immédiat ; fidélité testée au lieu d'être promise.
Scientifique/médical	VTK + ITK + DCMTK/GDCM + PDAL	Couvre volumes, DICOM, champs et nuages de points.
GPU	Kokkos ; workers CPU/CUDA/HIP, SYCL ensuite	Même source, binaires ciblés ; OpenCL n'est qu'un recours.
Rendu	VTK d'abord ; ANARI/Vulkan/OpenUSD en plugins	Réduit fortement le délai du MVP.
Backend CAE	MEDCoupling direct optionnel ; SALOME 9.16 via agent isolé	Aucun fork ni dépendance du cœur à KERNEL, CORBA, Qt ou au Python SALOME.
IA	DLPack + Arrow C Device ; PyTorch/ONNX Runtime externes	Évite de réimplémenter cuDNN et permet le partage mémoire mesuré.
HPC	Slurm + MPI du centre + Spack + Apptainer	ADIOS2 et UCX sont activés seulement lorsque le streaming ou le multinœud le justifie.
Licence	Apache-2.0 OR MIT pour le nouveau cœur	Double licence permissive ; l'intention non commerciale relève de la gouvernance, jamais d'une clause NC.

Verdict

Le besoin est réel, mais la demande pour un produit complet doit encore être validée. La première preuve inclut désormais SALOME comme banc de référence dès P0 : trajet reproductible STEP/DICOM/LAS → Morphoia IR → rendu/simulation/tenseur → résultat, plus un maillage SMESH/MED comparé sans introduire SALOME dans le cœur.

Statut, portée et mode d'emploi

Ce document constitue une **architecture de référence v0.2**. Il conserve les arbitrages structurants de la v0.1 et corrige le positionnement de SALOME : backend CAE optionnel et oracle différentiel dès P0, sans dépendance du cœur. Il ne constitue ni une spécification normative complète, ni une garantie de conformité d'un dispositif médical, ni un avis juridique. Les faits susceptibles d'évoluer ont été vérifiés auprès de documentations officielles le 6 août 2026.

Élément	Valeur
Porteur	Dr Olivier Ami
Projet	Morphoia Engine
Version du document	0.2
Date de référence	6 août 2026
Public cible	Contributeurs C++/Python, chercheurs, centres HPC, spécialistes CAO, imagerie et simulation
Hypothèse économique	Développement non commercial, collaboration ouverte, absence de dépendance obligatoire à un service cloud payant
Décision de licence	Apache-2.0 OR MIT pour le nouveau cœur ; documentation CC BY 4.0 ; marques et logo gérés séparément
Horizon	Spike SALOME P0 en ≈ 10 jours ; vertical slice en 4 mois ; v1 crédible en 15 à 18 mois

Niveaux de certitude

Niveau	Sens dans ce rapport	Exemple
Décidé	Arbitrage recommandé dès maintenant.	C++20, ABI C, Python, IR fédérée.
À valider	Choix retenu sous réserve d'un spike mesuré.	Kokkos + PETSc sur kernels représentatifs.
Option	Brique utile, mais non requise au MVP.	ANARI, OpenUSD/Hydra, SYCL, viewer web.
Hors périmètre	Promesse explicitement refusée.	Lecture parfaite de tous les formats propriétaires ; chargement direct des plugins de tous les moteurs.

Table des matières

Décision en une page	2
Statut, portée et mode d'emploi	3
Niveaux de certitude	3
Table des matières	4
1. Besoin et opportunité	7
1.1 Le problème n'est pas l'absence de bons logiciels	7
1.2 Le besoin réellement défendable	7
1.3 Fenêtre stratégique	7
1.4 Hypothèses qui doivent être validées	8
2. Paysage concurrentiel	9
2.1 Le concurrent le plus proche : SALOME	9
2.2 Matrice fonctionnelle	9
2.3 Concurrents spécialisés et briques à fédérer	9
2.4 Stratégie de coexistence	10
2.5 Avantage défendable de Morphoia	10
2.6 Profil d'intégration SALOME 9.16	11
3. Principes d'architecture	12
3.1 Principes non négociables	12
3.2 Décomposition des dépôts	12
3.3 Langages et contrats publics	13
4. Morphoia IR : le contrat de continuité	14
4.1 Ce que l'IR contient	14
4.2 Ce que l'IR ne contient pas	14
4.3 Sérialisation recommandée	15
4.4 Stockage adressé par contenu	15
4.5 Références externes SALOME	15
5. Ingestion, formats et représentations	16
5.1 Profils de prise en charge	16
5.2 CAO et formats propriétaires	16
5.3 Imagerie médicale	16
5.4 Maillages, points et volumes	17
6. Rendu et visualisation	18
6.1 Architecture de rendu progressive	18
6.2 Pourquoi ne pas choisir un seul moteur	18
6.3 LOD et streaming	18
7. Simulation et portabilité GPU	19
7.1 Kokkos comme modèle de programmation, pas comme promesse magique	19

7.2 Bibliothèques de calcul recommandées	19
7.3 Contrat de simulation	20
7.4 Politique numérique	20
8. Intelligence artificielle et échange sans copie	20
8.1 Frontière de responsabilité	20
8.2 Runtimes	20
8.3 CUDA/cuDNN et l'exigence "full open source"	21
8.4 Retour des prédictions	21
9. Plugins, extensions et connecteurs	21
9.1 Quatre niveaux d'extension	21
9.2 Manifeste d'extension	21
9.3 Compatibilité avec les plugins d'autres moteurs	21
9.4 Contrat de l'agent headless SALOME	22
9.5 Politique de stabilité	22
10. Exécution locale, cloud et EuroHPC	22
10.1 Job bundle immuable	22
10.2 Pile recommandée	22
10.3 Profils EuroHPC à valider tôt	23
10.4 Profils de site	23
11. Sécurité, confidentialité et qualité	24
11.1 Modèle de menace minimal	24
11.2 Données de santé	24
11.3 Secure software supply chain	24
12. Licence et gouvernance open source	24
12.1 Arbitrage de licence	24
12.2 Répartition des licences	25
12.3 Matrice de dépendances	25
12.4 Profil SALOME composant par composant	25
12.5 Quatre niveaux de distribution	26
12.6 Gouvernance à faible friction	26
13. Vertical slice et feuille de route	26
13.1 P0 — 6 semaines : spécifier et tuer les risques	27
13.2 P0-SALOME — environ 10 jours d'ingénierie	27
13.3 P1 — mois 2 à 4 : v0.1 démontrable	27
13.4 P2 — mois 5 à 9 : alpha multi-domaine	27
13.5 P3 — mois 10 à 14 : bêta EuroHPC	27
13.6 P4 — mois 15 à 18 : v1.0	28
14. Plan expérimental et critères de validation	28
14.1 Corpus progressif	28

14.2 Mesures de fidélité	28
14.3 Mesures de performance	28
14.4 Matrice de compatibilité CI	29
14.5 Suite de conformité différentielle SALOME	29
15. Charge, coûts et leviers de réduction	29
15.1 Estimation réaliste	29
15.2 Les économies les plus importantes	29
15.3 Ce que l'IA générative peut accélérer	29
15.4 Dépendances externes à obtenir	30
15.5 Effort marginal du profil SALOME	30
16. Registre de risques et décisions	30
16.1 Risques prioritaires	30
16.2 Architecture Decision Records initiaux	31
16.3 Décisions immédiates des 30 prochains jours	31
Annexe A. Pile logicielle de référence	32
Annexe B. Formats et niveaux de service proposés	33
Annexe C. Glossaire d'architecture	34
Annexe D. Profil technique SALOME 9.16	35
D.1 Frontière d'exécution et protocole	35
D.2 Mapping des artefacts et de Morphoia IR	36
D.3 Conformité, packaging et exploitation	37
Annexe E. Références officielles	38

01

Pourquoi Morphoia ?

Besoin, fenêtre d'opportunité et limites de la promesse

1. Besoin et opportunité

1.1 Le problème n'est pas l'absence de bons logiciels

L'écosystème dispose déjà d'excellentes briques : SALOME pour l'intégration CAO-maillage-solveurs, ParaView/VTK pour la visualisation scientifique, 3D Slicer pour l'imagerie médicale, SOFA pour la biomécanique interactive, FreeCAD/OCCT pour la CAO exacte, OpenUSD pour la composition de scènes, et de nombreuses piles de calcul et d'IA. Le problème persistant est la **rupture de continuité** entre ces mondes : changements d'unités ou de repères, pertes de métadonnées, conversions non traçables, copies mémoire, dépendance à un constructeur GPU et impossibilité de rejouer le même workflow sur un cluster.

Cette fragmentation est explicitement reconnue par l'Alliance for OpenUSD, dont la spécification Core 1.0 vise l'interopérabilité des écosystèmes 3D, ainsi que par les initiatives européennes consacrées aux jumeaux humains virtuels et aux workflows couplant simulation, données et IA. La Commission européenne a engagé plus de 100 M€ sur l'initiative Virtual Human Twins ; EDITH insiste sur une infrastructure fédérée de données, modèles, algorithmes et calcul ; interTwin a construit un prototype open source de Digital Twin Engine. [\[R02\]](#) [\[R12\]](#) [\[R13\]](#) [\[R11\]](#)

1.2 Le besoin réellement défendable

Une continuité vérifiable entre géométrie exacte, données discrètes, imagerie médicale, simulation, IA et HPC hétérogène — sans imposer un format, un solveur, un moteur de rendu ou un constructeur de GPU unique.

Besoin	Intensité	Réponse Morphoia
Nouveau moteur de rendu généraliste	Faible	Réutiliser VTK, ANARI, Vulkan et Hydra ; ne pas développer un moteur complet.
Nouvelle CAO paramétrique générale	Faible à moyenne	OCCT/FreeCAD comme moteurs externes ; Morphoia porte le contrat d'échange et la provenance.
Interopérabilité CAO-médical-simulation-IA	Forte	IR fédérée, registres de pertes, conversions validées, adaptateurs spécialisés.
Portabilité NVIDIA-AMD-Intel	Forte	Workers ciblés Kokkos et providers IA séparés ; CPU obligatoire.
Exécution reproductible EuroHPC	Forte	Job bundles, profils de sites, Spack/Aptainer/Slurm/ADIOS2.
Chargement universel de plugins tiers	Irréaliste	Adaptateurs et exécution dans l'application d'origine ou un processus isolé.
Lecture parfaite de tous les formats	Impossible	Profil de formats ouverts, connecteurs optionnels, rapport de fidélité.

1.3 Fenêtre stratégique

- La standardisation OpenUSD progresse, mais elle traite surtout la description/composition de scènes et ne remplace ni STEP/AP242, ni DICOM, ni les formats de calcul scientifique. [\[R02\]](#) [\[R31\]](#)
- EuroHPC demande le portage d'applications vers des accélérateurs hétérogènes et soutient les usages mêlant jumeaux numériques, IA et grands volumes de données. [\[R29\]](#) [\[R30\]](#)

- Les domaines médicaux cherchent une infrastructure fédérée de modèles et de données, mais les outils actuels restent principalement spécialisés par modalité ou application. [R12] [R13]
- La disponibilité de bibliothèques ouvertes et matures rend possible une plateforme d'intégration à coût raisonnable, à condition de refuser la réécriture des moteurs existants.

1.4 Hypothèses qui doivent être validées

- 1 Des équipes externes acceptent une couche supplémentaire si elle élimine effectivement des conversions manuelles et améliore la reproductibilité.
- 2 Le coût d'intégration d'un adaptateur est inférieur au coût d'entretien d'une chaîne point-à-point spécifique à chaque logiciel.
- 3 Les performances d'un chemin portable Kokkos/PETSc restent suffisamment proches des implémentations constructeur sur trois kernels représentatifs.
- 4 Les informations de fidélité et de provenance sont perçues comme une valeur opérationnelle, et pas seulement documentaire.
- 5 Les partenaires HPC acceptent le modèle de job bundle et les images Apptainer produites par le projet.

Go/No-Go marché

Avant d'élargir l'équipe, conduire 20 à 30 entretiens structurés : CAO/CAE, imagerie médicale, biomécanique, plateformes HPC et IA 3D. Le signal attendu n'est pas "l'idée est intéressante", mais l'engagement à fournir un cas test, installer le prototype ou contribuer un adaptateur.

2. Paysage concurrentiel

2.1 Le concurrent le plus proche : SALOME

SALOME 9.16 est traité comme un **backend CAE optionnel et un banc de référence**, non comme le socle de Morphoia. Morphoia utilise directement les bibliothèques amont lorsqu'elles suffisent, notamment OCCT et VTK/ParaView. Les capacités propres à SALOME, notamment SHAPER/GEOM, SMESH, YACS et JOBMANAGER, sont accessibles par un agent headless isolé. KERNEL, CORBA, Qt, les versions patchées des bibliothèques et l'environnement Python de SALOME restent confinés à ce runtime. SALOME apporte une chaîne CAO-maillage-champs-orchestration très mature, mais ni DICOM natif, ni data plane IA sans copie, ni portabilité GPU multi-constructeur structurée. [\[R01\]](#) [\[R48\]](#) [\[R55\]](#)

Cette distinction est composant par composant. MEDCoupling/MEDLoader se prête à un lien C++ ou Python autonome pour manipuler maillages, champs et transferts ; SMESH déclare au contraire KERNEL, GUI et GEOM parmi ses prérequis et demeure dans l'agent. SHAPER/GEOM fournit l'oracle CAO et les groupes ; ParaVis est le ParaView intégré ; YACS et JOBMANAGER ne sont que des hôtes de compatibilité conditionnels. L'architecture distribuée standard SALOME 9.16 utilise CORBA, tandis qu'un mode session-less peut éviter ses serveurs pour certaines API ; le spike P0 déterminera le mode minimal réellement compatible avec les opérations retenues. [\[R55\]](#) [\[R56\]](#) [\[R57\]](#) [\[R58\]](#) [\[R59\]](#)

2.2 Matrice fonctionnelle

Plateforme	CAO	Visu	Médical	Solveur	IA	Supervision	Rôle dominant
SALOME	Très fort	Très fort	Absent	Absent natif	Faible / absent	Très forte	Intégration et supervision CAE
ParaView / VTK	Faible	Très fort	Moyen	Absent	Moyen	Forte	Visualisation et post-traitement
3D Slicer	Faible	Fort	Très fort	Absent	Fort	Faible	Plateforme d'imagerie médicale
SOFA	Faible	Moyen	Moyen	Très fort	Moyen	Moyenne	Biomécanique interactive
FreeCAD / OCCT	Très fort	Moyen	Absent	Faible	Faible	Faible	CAO paramétrique et B-Rep
interTwin DTE	Faible	Faible	Faible	Via codes	Fort	Très forte	Workflows fédérés de jumeaux
NVIDIA Omniverse	Faible	Très fort	Moyen	Moyen	Très fort	Forte	Pile intégrée NVIDIA
OpenUSD / Hydra	Faible	Très fort	Absent	Absent	Moyen	Faible	Scènes, composition, rendu
Open3D	Faible	Fort	Absent	Absent	Très fort	Faible	Points, maillages, ML 3D
Blender / Godot	Faible	Très fort	Absent	Faible	Moyen	Faible	Création ou temps réel
Morphoia cible	Fort	Fort	Fort	Via providers	Très fort	Très forte	Continuité et traçabilité

Lecture : cette matrice qualifie la couverture native du socle examiné, pas la possibilité théorique d'ajouter un module. Pour SALOME, "médical absent" signifie absence d'un modèle DICOM clinique natif comparable à Slicer/DCMTK/GDCM ; "IA faible/absente" signifie absence documentée d'un data plane multi-framework sans copie et d'une portabilité GPU unifiée, même si la distribution 9.16 recense ONNX parmi ses prérequis. SALOME supervise des solveurs sans être lui-même un solveur général. [\[R08\]](#) [\[R48\]](#) [\[R55\]](#)

2.3 Concurrents spécialisés et briques à fédérer

Famille	Exemples	Force	Place recommandée
Solveurs HPC	PETSc, MFEM, FEniCSx	Algèbre, PDE, FEM et accélération	Plugins de calcul ; ne pas recréer leurs solveurs.
Biomécanique	SOFA, FEBio, SimVascular	Modèles et workflows de domaine	Adaptateurs de problèmes/résultats, hors processus si nécessaire.
Visualisation	ParaView/VTK, Slicer	Volumes, champs, segmentation, plugins	Interfaces principales du MVP.
Création/temps réel	Blender, Godot, O3DE	DCC, interaction, XR	Extensions dédiées via USD/gITF ; aucune promesse binaire universelle.

Ces outils confirment que les fonctions verticales existent déjà. Le manque porte sur le contrat transversal : identité des objets, unités, repères, pertes, provenance, mémoire accélérateur et rejouabilité d'un site à l'autre. [\[R51\]](#) [\[R52\]](#) [\[R53\]](#) [\[R54\]](#)

2.4 Stratégie de coexistence

Écosystème	Ne pas faire	Faire
SALOME — direct	Lier le cœur aux modules de plateforme.	Réemployer MEDCoupling/MEDLoader dans un plugin LGPL dynamique séparé ; conserver OCCT/VTK amont.
SALOME — agent	Partager objets CORBA, TopoDS_Shape ou pointeurs.	Exécuter SHAPER/GEOM/SMESH dans un runtime 9.16 isolé et transporter STEP/XAO/MED par URI/hash.
SALOME — oracle	Déclarer SALOME vérité automatique.	Comparer import, maillage, champs et ParaVis ; toute divergence déclenche une analyse qualifiée.
ParaView / VTK	Écrire un viewer scientifique complet.	Fournir lecteurs/écrivains Morphoia, filtres et représentation VTKHDF.
3D Slicer	Réimplémenter segmentation, DICOM et MRML.	Extension Slicer qui importe/exporte l'IR et déclare les transformations.
FreeCAD	Construire une nouvelle CAO interactive.	Workbench pour créer/inspecter des nœuds paramétriques Morphoia.
SOFA	Absorber son graphe de simulation dans le cœur.	Mapper géométrie, matériaux, conditions et résultats via un adaptateur.
OpenUSD	Faire d'USD l'autorité géométrique universelle.	L'utiliser pour la composition et la présentation dérivées.
Blender / Godot	Promettre la compatibilité binaire de leurs plugins.	Échanger via USD/gITF et fournir des extensions dédiées.

2.5 Avantage défendable de Morphoia

- Un objet peut conserver simultanément une B-Rep exacte, plusieurs tessellations, des champs, un volume, des tenseurs et des représentations MVX, reliés par provenance.
- Chaque opération expose unités, repères, tolérances, identifiants, version du convertisseur, pertes et réparations.
- Les mêmes job bundles s'exécutent localement, sur CUDA, sur HIP et sur Slurm sans changer la sémantique du workflow.
- Les échanges IA sont des contrats mémoire et de synchronisation, pas des exports de fichiers ad hoc.
- La plateforme reste utile même si l'utilisateur préfère SALOME, Slicer, ParaView, FreeCAD ou un notebook comme interface principale.

Positionnement à employer

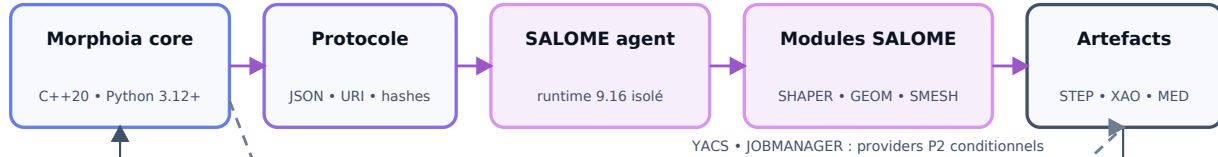
“Open interoperability fabric for 3D scientific computing” est plus exact que “universal 3D engine”. Le second promet un remplacement ; le premier décrit un contrat commun qui augmente la valeur des outils existants.

2.6 Profil d'intégration SALOME 9.16

RÉEMPLOI DIRECT

morphoia-io-med • MEDCoupling / MEDLoader
plugin dynamique LGPL séparé • C++ / Python

PROCESSUS EXTERNE



CAS / fichiers adressés par contenu • jamais de payload massif dans le plan de contrôle

ORACLE DIFFÉRENTIEL

morphoia-conformance-salome

comparaison • divergence → analyse • aucune vérité automatique

Brique	Décision Morphoia	Intégration	Phase
OCCT / XDE	Conserver l'utilisation directe amont.	Bibliothèque native.	Décidé
MEDCoupling / MEDLoader	Réemploi direct prioritaire, mais optionnel.	Plugin dynamique LGPL séparé.	P0-P1
SMESH	Réutiliser génération, groupes, hypothèses et contrôles qualité.	Agent hors processus ; MED en sortie.	P0-P1
SHAPER / GEOM	CAO paramétrique, provenance et oracle STEP.	Agent ; STEP/XAO/Python dump.	Spike P0 ; P2
ParaVis	Ne pas reprendre comme moteur.	Oracle ; ParaView/VTK amont reste principal.	P1
YACS	Compatibilité workflows SALOME existants.	Nœud "Morphoia job bundle".	P2 conditionnel
JOBMANAGER	Exécuteur compatible sites SALOME.	Provider optionnel ; Slurm direct autoritaire.	P2 conditionnel
KERNEL / CORBA / SALOMEDS	Ne jamais intégrer au cœur.	Confinés dans l'agent SALOME.	Exclu du cœur
Qt / PyQt	GUI SALOME seulement ; jamais dans le cœur.	Agent sans GUI ; le runtime SALOME peut les contenir.	Hors base
MeshGems	Exclu du profil open source.	Activation locale licenciée seulement.	Exclu par défaut

Frontière d'architecture

Aucun TopoDS_Shape, objet CORBA ou pointeur SALOME ne franchit la frontière. Les artefacts massifs passent par CAS/fichiers ; le protocole ne transporte que métadonnées, URI, hashes, statuts et diagnostics. La mention "SALOME 9.16 integration profile" n'implique aucune validation par CEA, EDF ou OPEN CASCADE.

02 Architecture cible

Un noyau fédéré, des moteurs spécialisés et des frontières stables

3. Principes d'architecture

Interfaces	Python • C ABI • CLI • notebooks
Orchestration	DAG • provenance • transactions • jobs
Morphoia IR	manifest • graphes • unités • pertes • identités
Représentations	B-Rep • mesh • points • voxels • champs • tenseurs
Moteurs spécialisés	OCCT • VTK/ITK • PDAL • Kokkos/PETSc • OpenUSD
Exécution	CPU • CUDA • HIP • SYCL • Slurm/EuroHPC

Les bibliothèques spécialisées restent remplaçables derrière des contrats stables.

3.1 Principes non négociables

Principe	Règle concrète	Pourquoi
Fédération	Une IR relie les représentations ; elle ne les aplatit pas.	Préserver la sémantique propre aux B-Rep, volumes et champs.
Autorité explicite	Chaque donnée indique sa source d'autorité et ses dérivés.	Éviter qu'une tessellation remplace silencieusement le modèle exact.
Headless first	Le cœur fonctionne sans GUI ni GPU d'affichage.	Indispensable pour CI, serveurs et HPC.
CPU baseline	Toute fonction essentielle possède un chemin CPU testable.	Ouverture, débogage, reproductibilité et absence de verrou constructeur.
Ports, pas wrappers	Les APIs Morphoia exposent des concepts stables, pas les types internes d'OCCT/VTK/Kokkos.	Limiter la propagation des ABI et des licences.
Pertes visibles	Aucune réparation ou conversion lossy sans journal et métrique.	Crédibilité scientifique et médicale.
Performance mesurée	Les gains incluent I/O, transferts, compilation et synchronisation.	Éviter les speedups artificiels limités au kernel.
Sécurité par frontière	Parseurs risqués et plugins non fiables sont isolés.	Les fichiers 3D et DICOM sont des entrées non fiables.

3.2 Décomposition des dépôts

Une organisation modulaire réduit le temps de compilation, clarifie les licences et permet aux utilisateurs de n'installer que ce dont ils ont besoin. Les modules lourds ne sont jamais requis par le paquet Python de contrôle.

Dépôt / paquet	Responsabilité	Dépendances fortes
morphoia-spec	Schémas IR, profil AP242, conventions, ADR, jeux de conformité.	Aucune runtime
morphoia-core	Handles, graphes, unités, provenance, CAS, diagnostics, transactions.	C++20, STL, JSON Schema
morphoia-capi	ABI C versionnée et SDK de plugins.	morphoia-core
morphoia-python	API Python, CLI, notebooks, orchestration locale.	ABI C, nanobind/pybind11
morphoia-io-*	Adaptateurs OCCT, VTK/ITK, PDAL, DICOM, USD, glTF.	Une pile spécialisée par module
morphoia-io-med	Maillages, champs, MED-file et remapping via MEDCoupling.	Plugin LGPL dynamique optionnel
morphoia-compute	Contrats mémoire, kernels portables, adaptateurs PETSc/MFEM.	Kokkos, option PETSc/MPI
morphoia-worker-*	Binares CPU/CUDA/HIP/SYCL ciblés.	Toolchain du backend
morphoia-hpc	Job bundles, profils Slurm, Spack, Apptainer, checkpoints.	YAML/JSON, Slurm client
morphoia-salome-protocol	Schémas versionnés : commandes, URI, hashes, capacités, pertes.	Aucune dépendance SALOME
morphoia-salome-bridge	Client Morphoia et agent headless exécuté dans SALOME 9.16.	Processus et distribution séparés
morphoia-conformance-salome	Corpus et comparaisons différentielles qualifiées.	CI jalonnée, sans runtime dans core
morphoia-connectors	FreeCAD, Slicer, ParaView, Blender, Godot.	APIs des hôtes, dépôts séparés

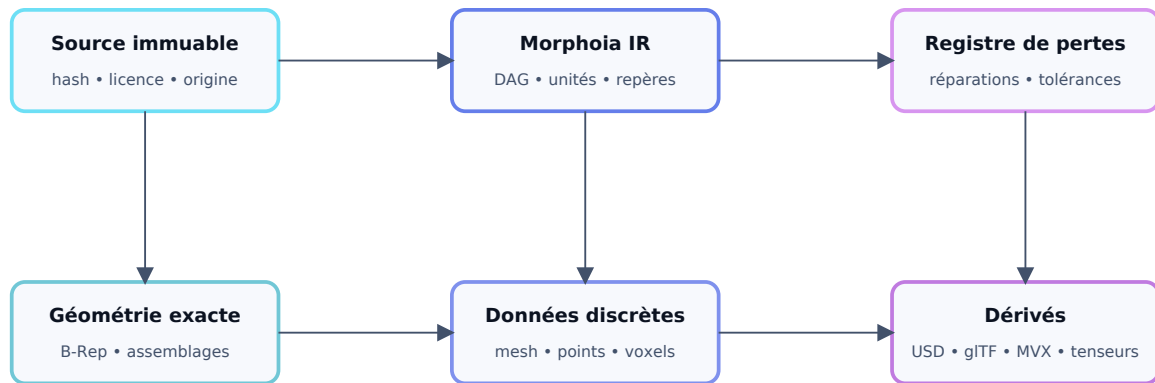
3.3 Langages et contrats publics

Couche	Choix	Arbitrage
Cœur natif	C++20	Meilleur dénominateur commun entre OCCT, VTK, ITK, Kokkos, MPI et solveurs HPC.
ABI stable	C11	Handles opaques, structures avec taille/version, allocateurs et erreurs explicites.
API utilisateur	Python 3.12+	Orchestration, recherche, notebooks ; appels lourds en natif.
Contrôle SALOME	JSON versionné + URI + hashes	Transport local simple ; messages petits ; ProbeCapabilities obligatoire.
Contrôle réseau futur	Protocol Buffers/gRPC optionnels	Seulement si un déploiement distant le justifie ; sémantique inchangée.
Données inter-processus	CAS/fichiers ; Arrow IPC/ADIOS2 ailleurs	SALOME échange STEP/XAO/MED ; jamais de payload massif dans le contrôle.
Build	CMake + presets	Standard natif commun ; Spack devient le gestionnaire officiel en HPC.

Le contrat C n'expose jamais `std::vector`, exceptions C++, RTTI, types OCCT/VTK/Kokkos ou pointeurs de bibliothèque. Il utilise des handles opaques, des vues mémoire décrites, des callbacks de durée de vie, des codes d'erreur structurés et une négociation de capacités. Cette discipline est plus importante que le choix entre pybind11 et nanobind.

Exemple de principe ABI : `struct_size` et `abi_version` en tête de toute structure extensible ; propriété de la mémoire documentée ; fonctions de destruction fournies par le même module qui alloue ; aucun état global caché ; diagnostics sérialisables.

4. Morphoia IR : le contrat de continuité



Aucune conversion ne remplace silencieusement l'autorité de la source.

4.1 Ce que l'IR contient

Bloc	Contenu minimal
Identité	UUID logique, hash SHA-256 du contenu, version du schéma, type MIME, taille.
Source	URI résolue, licence, auteur/détenteur, date, outil source, empreinte binaire.
Géométrie	Unités, repère, handedness, orientation, tolérances, bornes, dimension topologique.
Produit	Assemblages, instances, noms, couleurs, couches, propriétés et identifiants stables quand disponibles.
Représentations	Références typées vers B-Rep, mesh, points, voxels, champs, scène, tenseurs et MVX.
Provenance	DAG d'opérations avec outil, version, paramètres, temps, environnement, parents et graines.
Qualité	Validation, pertes, réparations, avertissements, métriques et seuils acceptés/refusés.
Sécurité	Classification, informations sensibles, politique d'accès, signature et état de confiance.

4.2 Ce que l'IR ne contient pas

- Les millions de points, voxels ou coefficients ne sont pas incorporés dans un JSON géant ; ils résident dans des blocs adressés par contenu.
- L'IR ne remplace pas le document STEP, le graphe DICOM, les fichiers VTKHDF ou les checkpoints ADIOS2.
- L'IR ne promet pas un identifiant topologique stable lorsque le moteur source n'en garantit pas ; elle enregistre la stratégie et son niveau de confiance.
- L'IR ne choisit pas une unique convention d'axes sans conserver la transformation depuis la convention d'origine.

4.3 Sérialisation recommandée

Usage	Format	Décision
Manifest humain et git	JSON canonique + JSON Schema 2020-12	Autorité v0.1 ; simple à inspecter, signer et générer en C++/Python.
API réseau	Protobuf/gRPC	Option après MVP ; contrôle seulement.
Tableaux en mémoire	DLPack + Arrow C Data/Device	Interopérabilité in-process, durée de vie et device explicites.
Tableaux persistants locaux	VTXHDF/HDF5, Nifti dérivé, Parquet/Arrow selon domaine	Pas de format unique : profils par type de données.
Chunks cloud/object store	Zarr v3 optionnel	Pour volumes et arrays distribués ; ne pas imposer au HPC POSIX.
Résultats temporels HPC	ADIOS2/BP5	Streaming, checkpoints, couplage in situ.

4.4 Stockage adressé par contenu

Le cache local et les bundles de job utilisent SHA-256 comme identifiant interopérable. Chaque bloc est immuable ; une transformation produit un nouveau bloc et un nouveau nœud de provenance. La déduplication, les reprises et la validation de transfert deviennent alors naturelles. Un hash rapide comme BLAKE3 peut être utilisé en interne pour le chunking, mais SHA-256 reste l'identité publiée afin d'éviter deux notions de vérité.

4.5 Références externes SALOME

Le bloc `external_refs.salome` conserve la provenance nécessaire sans transformer les identifiants SALOME en identités canoniques Morphoia. Un `study_entry` peut changer lors d'une copie ou d'une reconstruction ; il reste donc une référence externe qualifiée.

Sous-bloc	Contenu obligatoire
runtime	Version SALOME/modules, MED-fichier effectif, profil MED demandé, OS/image et hash.
source	Module SHAPER, GEOM, SMESH ou MEDCoupling ; étude et study_entry non canoniques.
geometry	Groupe, sous-maillages, associations géométriques, noms et stratégie de sélection.
mesh	Algorithmes, hypothèses, ordres, numérotation d'origine et table de renumérotation.
fields	Support nœuds/cellules/Gauss, composantes, unités, nature intensive/extensive, temps.
fidelity	Réparations, conversions, pertes, avertissements, tolérances et métriques.
replay	Hash du script ou Python dump SHAPER et références vers STEP/XAO/MED/Study HDF.

Règle de fidélité

Une étude SALOME .hdf est un artefact opaque de reproductibilité, pas l'identité de l'objet. En médical, DICOM et son Frame of Reference restent l'autorité : un maillage MED dérivé ne remplace jamais le graphe DICOM.

5. Ingestion, formats et représentations

5.1 Profils de prise en charge

Morphoia doit annoncer des **profils testés**, pas une liste marketing de formats. Chaque profil précise lecture, écriture, assemblages, attributs, unités, métadonnées, niveau de round-trip, limites et corpus de conformité. OCCT prend en charge STEP AP203/AP214 et certaines parties d'AP242, IGES, glTF et OBJ ; XDE transporte notamment noms, couleurs et couches. Il serait donc trompeur d'annoncer un AP242 universel sans profil et tests. [\[R06\]](#) [\[R07\]](#)

Domaine	Autorité / import	Dérivés	Priorité
CAO ouverte	STEP AP242 profil Morphoia, AP214, IGES, OCCT BREP	Tessellations, USD, glTF	MVP
Maillages	VTX/VTXHDF, STL, OBJ, PLY, Gmsh, XDMF selon adaptateur	LOD, champs, glTF	MVP
CAE / SALOME	MED pour maillages et champs ; XAO pour associations spécialisées	VTXHDF, IR, rapports de pertes	P0-P1
Étude SALOME	Study .hdf conservée opaque avec hash et versions	Aucun round-trip hors runtime qualifié	P0
Points	LAS/LAZ, COPC, EPT, PLY via PDAL	Octree/LOD, Arrow/Parquet	MVP
Médical	DICOM original + graphe de références	NIfTI/Zarr/VTXHDF, surfaces	MVP
Volumes creux	OpenVDB	NanoVDB GPU read-only	v0.2
Scènes	OpenUSD + MaterialX	glTF/GLB de diffusion	v0.2
Résultats HPC	ADIOS2/BP5, VTXHDF	ParaView, champs Morphoia	v0.2
MXV	Encodages LP3 X/Y/Z, variantes expérimentales	Tenseurs et reconstruction	Expérimental

5.2 CAO et formats propriétaires

CATIA, NX, Creo, SolidWorks, Parasolid, ACIS ou DWG ne peuvent pas être garantis par un cœur entièrement libre. La stratégie officielle est : (1) export depuis l'application d'origine vers STEP/AP242 ; (2) connecteur exécuté dans l'application source ; (3) SDK propriétaire optionnel installé par l'utilisateur ; (4) rapport de perte identique dans tous les cas. Aucun SDK propriétaire n'est livré dans la distribution open source.

SHAPER sait maintenir ses propres paramètres, groupes et sélections et produire un dump Python de l'état courant. Un STEP tiers reste toutefois une fonction d'import : ni la documentation ni le format STEP ne garantissent la reconstruction de l'historique paramétrique du logiciel CAO source. La faisabilité du dump depuis l'agent sans GUI est un résultat attendu du spike, pas une promesse préalable. [\[R59\]](#) [\[R65\]](#)

Niveau	Promesse autorisée	Exigence de test
A — round-trip	Lecture et réécriture sans perte au sein du profil.	Égalité sémantique et géométrique selon tolérances publiées.
B — fidèle	Géométrie et attributs utiles préservés, quelques constructions non réécrites.	Métriques et liste des entités dégradées.
C — visualisable	Tessellation ou volume affichable, sans garantie d'éditabilité.	Bornes, orientation, unités et qualité de rendu.
D — expérimental	Import possible mais non qualifié.	Aucun engagement ; diagnostics obligatoires.

5.3 Imagerie médicale

DICOM est un ensemble de parties couvrant objets, services, encodage, sécurité, rapports et services web, mis à jour plusieurs fois par an. Le projet doit conserver les fichiers et relations d'origine : Study/Series/SOP Instance UIDs, Frame of Reference, géométrie par frame, SEG, SR, RT, encapsulations et tags privés selon politique. Une simple conversion en NIfTI ou Zarr ne suffit pas comme autorité. DICOMweb fournit QIDO-RS, WADO-RS et STOW-RS pour une intégration moderne. [\[R44\]](#)

- DCMTK gère l'objet DICOM, les dictionnaires et les services ; ITK/GDCM gère efficacement les pixels et les volumes N-D.
- Les dérivés anonymisés ne sont créés qu'après application d'un profil explicite ; le lien vers la source sensible est protégé et audité.

- L'orientation patient, les espacements, les transforms et les unités sont testés par phantoms, pas uniquement par inspection visuelle.
- Morphoia v1 demeure un outil de recherche tant qu'aucun cycle réglementaire et aucune validation clinique ne sont conduits.

5.4 Maillages, points et volumes

VTK fournit les principaux modèles de données de visualisation scientifique et un traitement parallèle ; ITK est conçu pour le traitement, la segmentation et le recalage d'images N-D ; PDAL traite et traduit des nuages de points à l'aide de pipelines déclaratifs. MEDCoupling complète cette pile pour les maillages, champs, points de Gauss, séries temporelles et transferts entre maillages ; MEDLoader lit et écrit le format MED au-dessus de MED-fichier. OpenVDB reste dédié aux volumes 3D creux. [\[R03\]](#) [\[R04\]](#) [\[R05\]](#) [\[R33\]](#) [\[R56\]](#) [\[R60\]](#)

Représentation	Indexation / chunking	Attributs critiques
Mesh surfacique	Blocs de sommets/indices + LOD	Normales, UV, matériaux, groupes, orientation, watertightness.
Mesh volumique	Partitions par cellules	Type d'élément, connectivité, ordre, matériaux, conditions aux limites.
Nuage de points	COPC/octree/chunks	CRS, intensité, couleur, temps, classification, précision.
Volume dense	Bricks/chunks multirésolution	Spacing, direction cosines, origine, type, calibration.
Volume creux	Arbre OpenVDB/NanoVDB	Transform, background, active topology, semantic class.
Champ	Attaché à points/cellules/voxels	Composantes, unité, repère, temps, interpolation.

6. Rendu et visualisation

6.1 Architecture de rendu progressive

Étape	Technologie	Usage
MVP	VTK headless + ParaView	Maillages, volumes, champs, rendu distant, tests d'images.
v0.2	ANARI	API haut niveau pour renderers scientifiques interchangeables.
v0.2/v0.3	Vulkan	Viewport interactif, culling, picking et calcul proche du rendu.
v0.3	OpenUSD/Hydra	Composition de scènes, matériaux et render delegates externes.
Diffusion	glTF/GLB + vtk.js/WebGPU	Viewer web léger de dérivés, jamais autorité CAO.

VTK est une bibliothèque C++ BSD-3-Clause avec Python et traitement parallèle ; elle réduit immédiatement le coût du viewer scientifique. ANARI abstrait les détails de plateforme et de backend de rendu à haut niveau. Vulkan fournit un accès graphique et compute bas niveau, multiplateforme et explicite, mais son coût d'ingénierie ne se justifie pas pour le premier démonstrateur. [\[R03\]](#) [\[R16\]](#) [\[R15\]](#)

ParaVis n'est pas un backend Morphoia supplémentaire. Il s'agit de l'intégration de ParaView dans SALOME. ParaView/VTK amont reste le chemin de rendu et de post-traitement principal ; ParaVis sert uniquement d'oracle de structure, de champs et de rendu dans la suite différentielle SALOME. [\[R61\]](#)

6.2 Pourquoi ne pas choisir un seul moteur

Approche	Avantage	Limite décisive
Tout VTK	Très rapide à livrer, scientifique et médical.	Moins adapté à un viewport CAO moderne ou aux scènes riches.
Tout Vulkan	Contrôle et performance.	Coût élevé ; réécriture de volume rendering, picking, LOD et outils.
Tout OpenUSD/Hydra	Écosystème de scène et render delegates.	Ni autorité CAO, ni pipeline médical, ABI en évolution.
Tout Godot/Blender	Interface riche et écosystème.	Sémantique, runtime et plugins propres ; HPC headless non central.
Architecture Morphoia	Spécialisation par backend.	Nécessite des contrats et tests d'adaptateurs disciplinés.

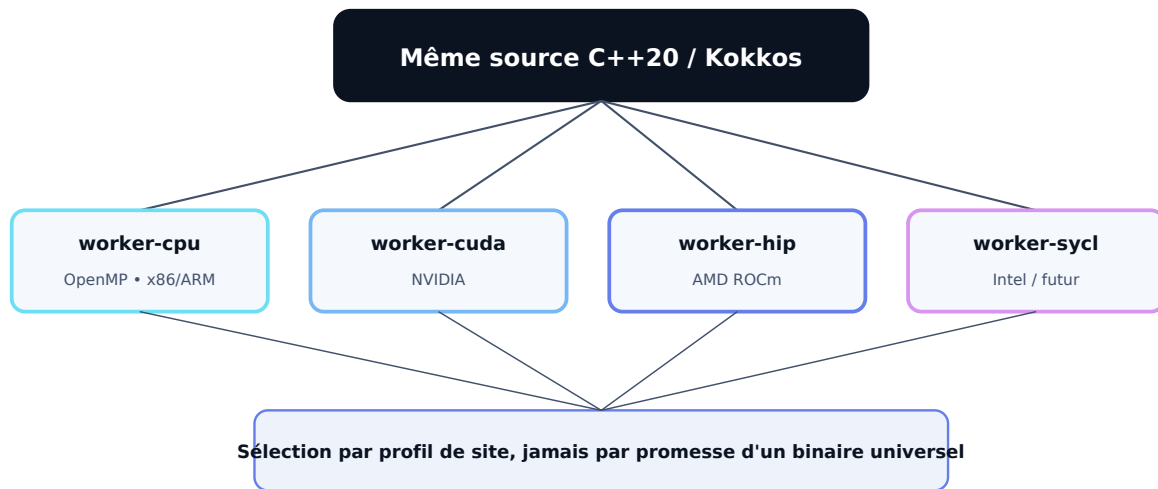
6.3 LOD et streaming

- B-Rep : tessellations dérivées par tolérance chordale/angulaire, versionnées et régénérables.
- Points : COPC/EPT ou octree interne, requêtes par frustum et budget de points.
- Volumes denses : bricks multirésolution, histogrammes pré-calculés et transfert progressif.
- Volumes creux : OpenVDB pour l'édition/traitement, NanoVDB pour le parcours GPU en lecture.
- Résultats HPC : streaming ADIOS2 ou lecture VTKHDF partitionnée ; le viewer ne télécharge jamais tout par défaut.

Arbitrage UI

Aucune application desktop propriétaire Morphoia au MVP. Livrer un CLI, des notebooks et des plugins ParaView/Slicer/FreeCAD. Une interface autonome n'est justifiée qu'après preuve d'un workflow que les hôtes existants ne savent pas présenter.

7. Simulation et portabilité GPU



7.1 Kokkos comme modèle de programmation, pas comme promesse magique

Kokkos fournit des abstractions C++ de parallélisme et de mémoire pour CUDA, HIP, SYCL, OpenMP et d'autres backends. Sa configuration ne permet qu'un backend device principal par build ; Morphoia doit donc publier des workers distincts, sélectionnés par profil de machine. Cette contrainte est compatible avec les partitions HPC homogènes et évite l'illusion d'un binaire universel. [\[R14\]](#)

Backend	Tier	Cible	Décision
CPU/OpenMP	1	x86-64 et ARM64	Toujours disponible ; référence numérique et CI.
CUDA	1	NVIDIA	Optimisation prioritaire ; bibliothèques constructeur optionnelles.
HIP/ROCm	1	AMD	Prioritaire pour LUMI et neutralité réelle.
SYCL	2	Intel et autres implémentations	Après stabilisation des kernels CPU/CUDA/HIP.
Vulkan compute	2	Kernels proches du rendu	Culling, voxelisation, post-traitement ; pas solveur général.
OpenCL	3	Matériel ancien/exotique	Compatibilité opportuniste seulement.

7.2 Bibliothèques de calcul recommandées

Besoin	Brique	Mode d'intégration
Kernels Morphoia	Kokkos + Kokkos Kernels	Source portable ; spécialisation constructeur si profilée.
Maillages CAE	SMESH	Fournisseur externe via agent ; MED et rapport de pertes en sortie.
Champs / remapping	MEDCoupling + INTERP_KERNEL	Plugin LGPL séparé ; transfert et conservation explicitement testés.
Algèbre/solveurs	PETSc	Adaptateur natif ; matrices/vecteurs device quand disponibles.
Éléments finis GPU	MFEM/libCEED	Plugin expérimental puis validé sur cas de référence.
Biomécanique interactive	SOFA	Processus/adapter de scène ; ne pas absorber le core.
Recherche PDE Python	FEniCSx	Job externe et import/export des maillages/champs.
CFD	OpenFOAM ou solveur choisi	Exécution externe ; licence copyleft isolée.
Algèbre constructeur	cuBLAS/cuSPARSE, rocBLAS/rocSPARSE, oneMKL	Option backend derrière Kokkos/PETSc ; pas dans l'ABI publique.

MEDCoupling expose en C++ et Python la manipulation des meshes/champs, tandis qu'INTERP_KERNEL et le remapper couvrent interpolation et localisation. SMESH reste hors processus et ne devient jamais un solveur. PETSc sait être configuré avec Kokkos et les backends CUDA, HIP, SYCL ou OpenMP ; MFEM propose plusieurs chemins accélérés. Tous ces choix restent soumis à des cas de référence mesurés. [\[R56\]](#) [\[R58\]](#) [\[R22\]](#) [\[R23\]](#)

7.3 Contrat de simulation

- Un **problem descriptor** référence géométrie, maillage, matériaux, conditions initiales/limites, solveur, précision et critères d'arrêt.
- Un **execution profile** choisit worker, précision, nombre de devices, MPI, partitionnement et politiques de mémoire.
- Un **result set** contient champs, scalaires, historique, résidus, logs, checkpoints, versions et métriques de performance.
- La sémantique du problème est indépendante du backend ; les options spécifiques sont namespacées et ne changent jamais silencieusement le modèle physique.

7.4 Politique numérique

Mode	Objectif	Règles
Référence	Validation	FP64 si pertinent, CPU, ordre déterministe, bibliothèques figées.
Portable	Résultat comparable	Tolérances par grandeur, réduction stable, mêmes critères d'arrêt.
Performance	Temps/coût minimal	Mixed precision autorisée, autotuning journalisé, validation a posteriori.
Exploratoire	Recherche rapide	Options libres mais marquées non qualifiées.

8. Intelligence artificielle et échange sans copie

8.1 Frontière de responsabilité

Morphoia expose des données et reçoit des prédictions ; il ne devient ni un framework de deep learning, ni un wrapper cuDNN, ni un gestionnaire d'entraînement distribué.

DLPack décrit les tenseurs denses stridés et leur device ; son protocole Python vise le partage sans copie lorsque cela est possible, sans le garantir dans tous les cas. Arrow C Data/Device fournit une ABI stable pour les données colonnaires, y compris en mémoire non CPU, et définit les mécanismes de synchronisation et de durée de vie. [\[R17\]](#) [\[R18\]](#)

Type	Contrat	Exemples
Tenseurs denses	DLPack versionné	Images, voxels, features, vertex arrays, embeddings.
Attributs colonnaires	Arrow C Data/Device	Points, labels, propriétés, résultats tabulaires.
Graphes/meshes	Vues d'indices + Arrow/DLPack	Edges, faces, cells et attributs associés.
Inter-processus	Arrow IPC/shared memory ou ADIOS2	Worker IA isolé ou entraînement HPC.
Persistance modèle	ONNX + fiche de modèle	Inférence portable et provenance.

8.2 Runtimes

Usage	Choix	Backends
Recherche/entraînement	PyTorch	CPU, CUDA, ROCm ; XPU selon environnement.
Inférence portable	ONNX Runtime	CPU, CUDA, TensorRT, MIGraphX, OpenVINO, WebGPU selon build.
Interopérabilité	DLPack	PyTorch, JAX, CuPy et runtimes compatibles.
Optimisation constructeur	Providers optionnels	Installés par profil ; jamais requis pour ouvrir un dataset.

ONNX Runtime choisit des Execution Providers au moyen d'une API commune ; le provider ROCm historique est déprécié et l'axe AMD recommandé est MIGraphX. Les packages combinant plusieurs providers exigent que leurs bibliothèques dépendantes soient présentes ; il faut donc reproduire la stratégie des workers ciblés. [\[R19\]](#)

8.3 CUDA/cuDNN et l'exigence "full open source"

Point juridique et technique

CUDA et cuDNN sont gratuits dans de nombreux usages, mais régis par des accords NVIDIA et ne constituent pas une pile intégralement open source. Le cœur Morphoia peut rester 100 % open source si ces éléments sont des backends optionnels installés par l'utilisateur ou le centre HPC. Une exécution CPU ouverte doit toujours exister ; ROCm est principalement open source mais ses composants ont des licences distinctes à auditer.

La licence cuDNN limite le SDK aux systèmes NVIDIA et précise les portions redistribuables ; la licence CUDA reste un EULA. AMD publie ROCm comme plateforme open source, tout en demandant un contrôle licence par composant. [\[R20\]](#) [\[R21\]](#)

8.4 Retour des prédictions

- Toute prédiction devient un champ, une annotation, une segmentation, un score ou une proposition de transformation avec provenance complète.
- Le modèle, les poids, le commit, le prétraitement, la précision, le device, la graine et les incertitudes sont enregistrés.
- Une sortie IA ne modifie jamais directement l'autorité géométrique ; elle crée une branche révisable et acceptée explicitement.
- Les copies mémoire, synchronisations et conversions dtype/layout sont instrumentées et visibles dans le profil de performance.

9. Plugins, extensions et connecteurs

9.1 Quatre niveaux d'extension

Niveau	Mécanisme	Usage	Confiance
Natif	ABI C + bibliothèque partagée	I/O et kernels haute performance	Signé et testé
Python	Entry points + API Python	Recherche, orchestration, petits traitements	Environnement contrôlé
Processus	gRPC/CLI + Arrow/ADIOS2	GPL, crash-prone, solveurs et outils externes	Isolé
WASI	WebAssembly sandboxé	Petites transformations CPU non fiables	Sandbox

9.2 Manifeste d'extension

Chaque extension déclare : identifiant, version sémantique, ABI requise, architectures, OS, formats, capacités, licences SPDX, dépendances, backends GPU, accès réseau/fichiers, isolation, niveau de fidélité, ressources maximales, signatures et tests de conformité. Le chargement repose sur les capacités et non sur le nom d'un plugin.

9.3 Compatibilité avec les plugins d'autres moteurs

Un plugin Blender dépend de l'API Python et du modèle de données Blender ; un GDExtension dépend de l'API C de Godot ; un plugin ParaView dépend de VTK/ParaView ; un render delegate dépend d'Hydra. Ils ne sont donc pas binaires interchangeables. Godot utilise précisément un pont C interne pour stabiliser GDExtension ; ParaView distingue plugins C++ et filtres Python ; Hydra dispose de son propre registre de renderers. [\[R34\]](#) [\[R36\]](#) [\[R37\]](#)

Connecteur	Mécanisme	Priorité	Valeur
ParaView	Reader/writer + filtres	P1	Visualisation immédiate des meshes, champs et volumes.
FreeCAD	Workbench Python/C++	P1	Création/inspection de B-Rep et opérations paramétriques.
3D Slicer	Extension Python/C++	P1	DICOM, segmentation et workflows médicaux.
SALOME	Agent headless + CAS/URI	P0-P1	SHAPER/GEOM/SMESH ; oracle différentiel.
OpenUSD/Hydra	Schema/API adapter	P2	Scènes et render delegates.
Blender	Extension Python + USD/glTF	P3	Création de contenu et contrôle visuel.
Godot	GDExtension + glTF/stream	P3	Viewer interactif et XR.

9.4 Contrat de l'agent headless SALOME

morphoia-salome-agent est lancé dans l'environnement Python et bibliothèques de SALOME 9.16. Il ne fournit aucune ABI au cœur : son protocole de contrôle est versionné, sérialisable et volontairement étroit.

Opération	Contrat minimal
ProbeCapabilities	Versions effectives, modules, formats MED, maillages OSS, limites et licences détectées.
Submit	Job id, opération, paramètres, URI/hash des entrées, budget temps/mémoire, sorties attendues.
Observe / Cancel	État, progression, logs structurés, diagnostics, annulation idempotente et timeout.
Publish	Manifest de sorties STEP/XAO/MED/HDF, tailles, hashes, unités, pertes et métriques.
Health	Runtime prêt, absence de GUI obligatoire, espace temporaire, version de protocole.

- Aucun TopoDS_Shape, objet CORBA, handle SALOME ou pointeur ne traverse le processus.
- Les artefacts massifs, y compris ceux dépassant 2 Gio, sont transportés par URI/hash et jamais dans CORBA, JSON ou gRPC.
- Les commandes sont idempotentes lorsque possible ; chaque tentative produit logs, environnement et statut terminal explicite.
- L'agent est lancé sans GUI ; l'installation SALOME peut contenir Qt/PyQt, mais le cœur ne les charge jamais. MeshGems reste absent du profil OSS.

9.5 Politique de stabilité

- ABI C majeure figée pendant un cycle v1.x ; ajout de champs en fin de structures dimensionnées.
- API Python suit SemVer, avec une période de dépréciation d'au moins deux versions mineures.
- Les adaptateurs vers des ABI instables (Hydra, hôtes desktop) vivent dans des dépôts séparés et des matrices de versions explicites.
- Les plugins de formats s'exécutent en processus isolé par défaut lorsqu'ils analysent des données complexes non fiables.

10. Exécution locale, cloud et EuroHPC

10.1 Job bundle immuable

Le worker HPC est headless et stateless. Il reçoit un bundle signé comprenant manifeste IR, hashes des données, commande, ressources, worker requis, versions, politique numérique, graines, emplacement des checkpoints et destinations de résultats. Aucune connexion à un service Morphoia central n'est nécessaire.

```
{ job, inputs[], worker, resources, numerical_policy, environment_lock, outputs, checkpoints, provenance }
```

10.2 Pile recommandée

Fonction	Brique	Rôle Morphoia
Ordonnancement	Slurm	Générer sbatch/srun ; ne pas écrire un scheduler.
Parallèle distribué	MPI + UCX	GPU-aware selon la pile du centre ; jamais embarquer un MPI arbitraire.
I/O et streaming	ADIOS2	Résultats, checkpoints, couplage et in situ.
Builds	Spack	Concrétisation par architecture, compilateur et backend ; lockfile.
Conteneurs	Apptainer	Image applicative ; drivers et souvent MPI fournis par le centre.
Compatibilité workflow	YACS	Provider optionnel : nœud consommant un Morphoia job bundle.
Compatibilité exécution	JOBMANAGER	Provider optionnel lorsque le site/utilisateur l'exige.
Objets/caches	POSIX, S3 compatible ou Data Lake du centre	Adaptateur de stockage ; hashes identiques.
Provenance	Morphoia IR + logs structurés	Identité du job, site, nœuds, versions, coûts et résultats.

Slurm est un gestionnaire de clusters open source et scalable. Apptainer expose CUDA, ROCm et d'autres accélérateurs aux conteneurs. Spack produit des environnements concrétisés et lockfiles. ADIOS2 fournit une API I/O/streaming conçue pour l'extrême échelle. UCX sait traiter des pointeurs mémoire CUDA et ROCm. YACS et JOBMANAGER savent héberger respectivement des graphes SALOME et des jobs/ressources, mais leur validation reste spécifique au site. [\[R24\]](#) [\[R25\]](#) [\[R26\]](#) [\[R27\]](#) [\[R28\]](#) [\[R66\]](#) [\[R67\]](#)

Autorité d'orchestration

Le DAG Morphoia et les bundles Slurm restent la définition autoritaire du travail. YACS et JOBMANAGER hébergent ou relaient ces bundles pour compatibilité ; ils ne deviennent ni le schéma canonique du workflow, ni une dépendance obligatoire.

10.3 Profils EuroHPC à valider tôt

Système	Architecture	Worker	Test prioritaire
LUMI	x86-64 + AMD MI250X/ROCm	HIP	Portabilité, I/O parallèle et scaling.
Leonardo	x86-64 + NVIDIA A100	CUDA	Comparaison CUDA/HIP et multi-GPU.
JUPITER	ARM64 Grace + NVIDIA GH200	CUDA ARM64	Cross-compilation, mémoire unifiée et scaling.

Les guides techniques EuroHPC décrivent notamment LUMI avec MI250X et Leonardo avec A100 ; JUPITER est le premier système exascale européen. Les appels Development Access sont ouverts en continu avec coupures mensuelles et un délai d'accès annoncé de quelques semaines. [\[R29\]](#) [\[R30\]](#) [\[R45\]](#)

10.4 Profils de site

Champ	Exemples
Scheduler	partition, account, qos, walltime, constraints, GRES.
Architecture	CPU target, GPU architecture, interconnect, mémoire, filesystems.
Software	modules, compiler, MPI, ROCm/CUDA, Apptainer, Spack upstream.
I/O	scratch, project, burst buffer, object store, quotas, purge policy.
Réseau	nœuds sans egress, proxy, data transfer nodes, registries autorisés.
Sécurité	images signées, chiffrement, données de santé, rétention, audit.

Règle de portabilité HPC

Le conteneur ne doit pas embarquer aveuglément drivers et MPI. Morphoia fournit une recette reproductible et un profil de site ; le centre conserve l'autorité sur les bibliothèques proches du matériel. YACS/JOBMANAGER restent optionnels.

03

Ouverture et confiance

Sécurité, licences, gouvernance et reproductibilité

11. Sécurité, confidentialité et qualité

11.1 Modèle de menace minimal

Menace	Mesure
Fichier malformé / décompression bomb	Limites de taille, temps et récursion ; parseur en processus ; fuzzing.
Plugin malveillant	Manifest de permissions, signature, sandbox/processus, réseau désactivé par défaut.
Dépendance compromise	SBOM SPDX/CycloneDX, pinning, signatures, scans, provenance de build.
Exfiltration de données médicales	Aucun réseau implicite, profils DICOM de-identification, chiffrement et audit.
Confusion de repères/unités	Types dimensionnés, conversions explicites, invariants et phantoms.
Résultat non reproductible	Versions, seeds, backend, précision, flags, environnement et tolérances capturés.
Output IA pris pour vérité	Provenance du modèle, incertitude, branche non autoritaire, validation humaine.

11.2 Données de santé

- Le cache de sources DICOM identifiantes est séparé des dérivés et chiffré au repos lorsque requis.
- Les profils de désidentification sont versionnés et testés ; les tags privés sont supprimés par défaut sauf justification.
- Les UUIDs peuvent être remappés de façon déterministe dans un domaine contrôlé, tout en conservant les relations internes.
- Les exports vers cloud ou HPC vérifient la classification des données et nécessitent une destination autorisée.
- Les journaux évitent les noms, identifiants et chemins contenant des données personnelles.

11.3 Secure software supply chain

- REUSE/SPDX pour toutes les sources ; SBOM générée à chaque release.
- Builds hermétiques autant que possible, dépendances épinglées, checksums vérifiés, releases signées.
- Sanitizers et fuzzers sur parseurs ; tests différentiels entre lecteurs lorsque possible.
- CI publique CPU sur chaque pull request ; GPU sur branches d'intégration et machines partenaires.
- Politique de sécurité et canal privé de divulgation avant la v0.1 publique.

12. Licence et gouvernance open source

12.1 Arbitrage de licence

Décision recommandée

Apache-2.0 OR MIT pour le nouveau code Morphoia Engine. Le choix Apache apporte une concession explicite de brevets ; l'option MIT maximise la compatibilité, notamment avec des intégrateurs GPL-2.0-only. Si le code existant est déjà sous MIT avec plusieurs auteurs, conserver MIT tant que tous les titulaires n'ont pas consenti au double régime.

Apache 2.0 accorde notamment un droit de brevet mondial, gratuit et irrévocable sous conditions ; MIT fournit une voie très courte et largement intégrable. L'intention "non commerciale" ne doit pas être ajoutée à la licence : la définition Open Source interdit de restreindre l'usage à un domaine, y compris les activités commerciales. Morphoia peut rester un projet gratuit, sans offre commerciale et gouverné dans l'intérêt commun, tout en permettant les usages commerciaux requis par l'open source. [\[R41\]](#) [\[R42\]](#)

12.2 Répartition des licences

Actif	Licence recommandée	Motif
Code Morphoia Engine	Apache-2.0 OR MIT	Brevet explicite via Apache ; voie MIT pour compatibilité maximale.
Schémas et spécifications	Apache-2.0 OR MIT	Autoriser des implémentations indépendantes.
Documentation	CC BY 4.0	Réutilisation avec attribution.
Exemples de code	Apache-2.0 OR MIT	Copie directe sans ambiguïté.
Jeux de test créés	CC0 ou CC BY 4.0	Réutilisation scientifique et benchmarks.
Poids de modèles	Licence dédiée compatible avec les données	Ne pas supposer que la licence du code s'applique aux poids.
Logo et nom	Marque / politique d'usage	Éviter les distributions trompeuses sans fermer le code.

12.3 Matrice de dépendances

Catégorie	Exemples	Politique
Permissive standard	VTk BSD-3, ITK Apache-2, Kokkos Apache-2 + LLVM, ADIOS2 Apache-2	Liens natifs autorisés ; notices et SBOM.
Licence custom	OpenUSD sous TOST-1.0 ; Slicer sous licence BSD-style propre	Plugins optionnels, LicenseRef et revue du paquet exact.
LGPL + exception/dynamique	OCCT LGPL-2.1 + exception, Qt/SOFA selon modules	Lien dynamique, notices, sources correspondantes et possibilité de remplacement.
GPL / copyleft fort	Certains solveurs, modules et plugins	Processus séparé ou distribution distincte ; revue juridique avant bundle.
SDK propriétaire	CUDA/cuDNN, importeurs CAO commerciaux	Option utilisateur/site ; jamais prérequis du cœur ni inclus par défaut.
Données tierces	Objaverse et corpus médicaux	Licence et consentement au niveau de chaque asset ; pas d'héritage implicite.

OCCT est sous LGPL-2.1 avec exception ; VTK sous BSD-3-Clause ; OpenVDB 12+ sous Apache-2.0 ; Kokkos actuel sous Apache-2.0 avec exception LLVM. OpenUSD utilise la licence personnalisée TOST-1.0. SALOME déclare globalement la plateforme sous LGPL v2.1, mais une distribution complète est une composition multilicence incluant prérequis, modules et variantes patchées. L'audit porte toujours sur l'artefact concret. [\[R39\]](#) [\[R40\]](#) [\[R33\]](#) [\[R46\]](#) [\[R47\]](#) [\[R48\]](#) [\[R62\]](#)

12.4 Profil SALOME composant par composant

Composant / artefact	Licence ou régime	Décision de redistribution
Protocole et bridge Morphoia	Apache-2.0 OR MIT	Code original indépendant ; aucune copie de code SALOME.
MEDCoupling	LGPL-2.1-or-later	Paquet dynamique séparé ; modifications et obligations LGPL conservées.
MED-fichier	LGPL-3.0-or-later	Dépendance de MEDLoader/HDF5 ; notices de la version exacte.
KERNEL, SHAPER, SMESH	LGPL-2.1-or-later dans les dépôts audités	Runtime distinct ; modules et meilleurs tiers audités séparément.
SALOME 9.16 complet	Composition multilicence	Recette d'installation d'abord ; image seulement après audit complet.
PyQt / PyQt-Charts	GPL-3.0 ou licence commerciale	GUI facultative séparée ; aucune dépendance du cœur.
MeshGems	Propriétaire commercial	Aucun binaire/plugin dépendant dans le profil OSS ; activation locale licenciée.
Image SALOME redistribuée	Obligations cumulées	Artefact distinct : SBOM, textes/notices, sources exactes, patches et relinkage si requis.

Frontière, pas exemption

La séparation en processus protège l'ABI et réduit le couplage. Elle ne supprime aucune obligation attachée à la copie ou à la redistribution. Pour P0-P1, Morphoia publie de préférence une recette reproductible et utilise l'installation SALOME 9.16 fournie par l'utilisateur ou le site.

La GPL autorise l'usage commercial. Une licence Riverbank commerciale devient pertinente lorsque le programme ou son mode de distribution n'est pas compatible avec la GPL ; il ne faut pas transformer cette alternative en interdiction générale. [\[R62\]](#) [\[R63\]](#) [\[R64\]](#)

12.5 Quatre niveaux de distribution

Artefact	Contenu	Règle
morphoia-core-oss	Cœur, ABI, CPU, Kokkos, VTK/ITK, DLPack/Arrow, PETSc/MFEM	Dépendances OSI permissives ; aucun runtime constructeur embarqué.
morphoia-lgpl-pack	OCCT et composants LGPL explicitement choisis	Bibliothèques remplaçables, sources correspondantes et notices.
morphoia-external-adapters	Slurm, SALOME, FreeCAD, Slicer, SOFA, outils GPL	CLI, API distante ou processus séparé ; pas de propagation dans l'ABI du cœur.
morphoia-vendor-recipes	CUDA, cuDNN, ROCm, oneMKL, SDK CAO	Détection et recettes seulement ; installation/licence assumée par l'utilisateur ou le site.

12.6 Gouvernance à faible friction

- DCO (Developer Certificate of Origin) plutôt qu'un CLA lourd au démarrage ; réévaluer si une fondation l'exige.
- Architecture Decision Records publics ; décisions techniques justifiées et réversibles.
- Roadmap, backlog, réunions et critères de release publics ; aucun composant essentiel privé.
- Mainteneurs par domaine et règles de revue : géométrie, médical, GPU, HPC, sécurité, licences.
- Conformance suite indépendante de l'implémentation afin de permettre d'autres moteurs compatibles.

La conformité REUSE et les identifiants SPDX rendent les licences lisibles par machine et simplifient la génération d'une nomenclature logicielle. [\[R43\]](#)

04

Réalisation

MVP, validation, coûts, risques et décisions immédiates

13. Vertical slice et feuille de route



Chaque phase est conditionnée par des critères Go/No-Go mesurables.

13.1 P0 — 6 semaines : spécifier et tuer les risques

Livrable	Contenu	Go/No-Go
IR v0.1	JSON Schema, unités, repères, provenance, pertes, CAS.	Rejouer 20 graphes et valider les schémas.
Import spike	STEP/AP242, série DICOM, LAS/LAZ.	Orientation/unités exactes ; erreurs structurées.
Compute spike	Même kernel CPU/CUDA/HIP.	Résultats dans tolérance et speedup bout-en-bout documenté.
IA spike	VTK/array → DLPack → PyTorch → champ.	Copie et sync instrumentées ; durée de vie sûre.
HPC spike	Job bundle Slurm + Apptainer sur un site.	Reprise checkpoint et provenance complète.
SALOME spike	Agent 9.16 headless, STEP, SMESH/MED, groupes et champs.	Zéro dépendance du core ; pertes/unités explicites.
MVX micro-corpus	30 à 50 objets représentatifs.	Pipeline stable avant tout corpus de 1 000 objets.

13.2 P0-SALOME — environ 10 jours d'ingénierie

Essai	Résultat attendu
Runtime	Lancer SALOME 9.16 sans GUI ; évaluer session-less puis session terminal standard.
Capacités	Implémenter ProbeCapabilities : versions, modules, MED, maillages OSS, limites et licences.
CAO	Comparer le même STEP par OCCT direct et SHAPER/GEOM ; mesurer unités, placements et topologie.
Maillage	Créer un maillage SMESH libre, groupes et hypothèses ; exporter en MED sans MeshGems.
Champs	Relire MED par MEDCoupling/MEDLoader : vecteurs, cellules/nœuds/Gauss et pas de temps.
Gros artefact	Tester >2 Gio exclusivement par URI/hash ; aucun payload massif dans le contrôle/CORBA.
Rejeu	Tester STEP/XAO/Study HDF et faisabilité d'un dump SHAPER depuis l'agent ; ADR + SBOM.

Go / No-Go P0-SALOME

Go si morphoia-core ne charge aucune dépendance SALOME/Qt/PyQt/omniORB, si aucune conversion d'unité ou de dimension n'est silencieuse, et si tous les groupes du profil déclaré sont conservés. L'absence de dump SHAPER headless n'arrête pas le bridge : elle dégrade seulement la stratégie de replay et doit être documentée.

13.3 P1 — mois 2 à 4 : v0.1 démontrable

- CLI et API Python documentées ; imports STEP/DICOM/LAS et exports dérivés VTK/gITF.
- morphoia-io-med, agent GEOM/SMESH 9.16 Linux x86-64, XAO/MED → IR et rapports de pertes.
- Viewer ParaView/VTK, rapports de perte et tableau de provenance.
- Workers CPU/CUDA/HIP pour trois kernels : transformation/échantillonnage, réduction sur champ, opérateur creux simple.
- Pont DLPack/Arrow avec PyTorch et ONNX Runtime CPU + un provider GPU.
- Bundle Slurm reproductible, Spack lock, image Apptainer et checkpoint ADIOS2.
- Validation MVX sur environ 1 000 objets Objaverse stratifiés, seulement après succès du micro-corpus.

13.4 P2 — mois 5 à 9 : alpha multi-domaine

- Connecteurs FreeCAD, Slicer et ParaView ; appels SHAPER/SMESH bidirectionnels qualifiés.
- Module GUI SALOME facultatif ; nœud Morphoia dans YACS ; JOBMANAGER seulement pour un cas utilisateur réel.
- Maillages volumétriques, champs temporels, VTKHDF et OpenVDB.
- PETSc et MFEM sur benchmarks définis ; partitionnement et MPI GPU-aware.
- OpenUSD/MaterialX pour les scènes dérivées ; ANARI comme renderer haut niveau optionnel.
- SDK de plugins C v1-rc, signatures, isolation de parseurs et conformance suite publique.

13.5 P3 — mois 10 à 14 : bêta EuroHPC

- Validation LUMI et Leonardo ; JUPITER ARM64 si accès disponible.
- Scaling fort/faible, I/O ADIOS2, profils de sites et reprise après interruption.
- Backend SYCL Tier 2 sur hardware partenaire ; comparaison des tolérances et performances.
- Audit sécurité et licences de la distribution ; SBOM et builds signés.
- Trois cas utilisateurs externes reproductibles et documentation d'exploitation.

13.6 P4 — mois 15 à 18 : v1.0

- ABI C v1 gelée, migration documentée, politique de compatibilité et support à long terme.
- Matrice de formats publiée avec niveaux A-D et corpus de conformité.
- Installations reproductibles Linux x86-64/ARM64 ; Windows Tier 2 pour viewer/import local.
- Benchmarks publics, gouvernance de mainteneurs et cycle de release semestriel.
- Décision argumentée sur l'utilité d'une interface Morphoia autonome.

14. Plan expérimental et critères de validation

14.1 Corpus progressif

Palier	Taille	But	Condition de passage
Smoke	10-20 assets	Déboguer les chemins critiques.	Aucun crash, diagnostics utilisables.
Faisabilité	30-50 assets stratifiés	Valider schémas, repères et round-trip.	Seuils géométriques atteints sur chaque famille.
Validation MVX	≈ 1 000 Objaverse	Mesurer reconstruction, invariance et coût.	Distribution d'erreurs stable ; cas d'échec catégorisés.
Interopérabilité	100 cas CAO + 100 DICOM + 100 points/meshes	Tester profils ouverts et edge cases.	Taux de succès et perte publiés.
Robustesse	10 000+	Fuzz-like corpus et performance out-of-core.	SLO de crash/mémoire/temps.
JEPA	Corpus Morphoia 3D complet	Entraînement représentationnel.	Seulement après qualification de l'encodage.

14.2 Mesures de fidélité

Domaine	Mesures obligatoires
CAO/B-Rep	Comptes topologiques, validité, distances, aire/volume, assemblages, PMI du profil, couleurs/couches.
Mesh	Hausdorff/Chamfer normalisés, normales, manifold/watertight, self-intersections, qualité éléments.
Points	Compte, CRS, attributs, précision, couverture, quantiles de distance et densité.
DICOM/volumes	UID relations, Frame of Reference, orientation, origine, spacing, intensités, frames manquantes.
Champs	Normes L2/L ∞ , conservation, unités, interpolation, pas de temps, précision numérique.
MVX	Chamfer/Hausdorff/F-score, IoU voxel/SDF, normales, topologie, perceptuel multi-vues, reconstruction utile pour IA.

14.3 Mesures de performance

- Temps vers premier objet visible et temps total d'import ; débit en Mo/s et éléments/s.
- Mémoire CPU/GPU maximale, allocations, fragmentation, taille des dérivés et taux de cache hit.
- Temps kernel, transferts, synchronisations, compilation/JIT, I/O et orchestration séparés.
- Speedup bout-en-bout CPU→GPU, coût énergétique si métriques du centre disponibles.
- Scaling fort/faible, efficacité parallèle, overhead MPI, débit de checkpoint et temps de reprise.
- Part des échanges IA réellement zero-copy et causes de copie résiduelle.

14.4 Matrice de compatibilité CI

Tier	Plateformes	Cadence
Tier 1	Linux x86-64 CPU, CUDA, HIP ; Linux ARM64 CPU/CUDA	Chaque merge ou nightly selon coût GPU.
Tier 2	Windows x86-64 CPU/Vulkan ; Linux SYCL	Hebdomadaire et avant release.
Tier 3	macOS arm64, OpenCL, WebGPU	Best effort ; pas de blocage v0.x.
HPC	LUMI, Leonardo, JUPITER ou équivalents	Jalons P2/P3 et release candidates.

14.5 Suite de conformité différentielle SALOME

Corpus / contrôle	Critère d'acceptation
20 STEP/AP242	Unités, placements, noms, couleurs, topologie, aire et volume comparés par invariants.
20 maillages SMESH	Types/ordres de cellules, connectivité et appartenances aux groupes conservés.
10 fichiers MED	Champs nodaux, cellulaires et Gauss ; composantes, unités et séries temporelles.
3 artefacts > 2 Gio	Aucune donnée massive dans le plan de contrôle ; URI/hash/taille vérifiés.
ParaView ↔ ParaVis	Même structure de champs ; divergences de lecteur/rendu qualifiées, pas masquées.
Isolation	Zéro dépendance SALOME, Qt, PyQt ou omniORB dans le cœur ; aucun MeshGems dans l'OSS.
Performance	Overhead chaud <15 % sur opérations longues ; démarrage mesuré et publié séparément.

Principe statistique

SALOME est un oracle différentiel : une divergence déclenche une analyse par invariants et, si nécessaire, un troisième outil. SALOME n'est jamais automatiquement considéré comme la vérité. Publier distributions, intervalles de confiance et catégories d'échec.

15. Charge, coûts et leviers de réduction

15.1 Estimation réaliste

Phase	Calendrier	Effort cumulé	Équipe type
P0	6 semaines	≈ 6 pers.-mois	Architecte C++/HPC, ingénieur géométrie/I/O, Python/QA à temps partiel.
P1 v0.1	4 mois	≈ 16 pers.-mois	3 à 4 FTE + expertise médicale/CAO ponctuelle.
P2 alpha	9 mois	≈ 28 pers.-mois	4 FTE, contributeurs adaptateurs.
P3 bêta	14 mois	≈ 40 pers.-mois	4 à 5 FTE + partenaires HPC.
P4 v1	15-18 mois	≈ 60 pers.-mois	4 à 5 FTE moyens, revue sécurité/licence et domaines.

Ces ordres de grandeur supposent une forte réutilisation des bibliothèques proposées et trois vertical slices, pas la promesse de remplacer des suites entières. Ils excluent la qualification réglementaire médicale et le développement d'un solveur multiphysique généraliste.

15.2 Les économies les plus importantes

Décision	Économie	Ce qui est sacrifié
Pas de GUI propre au MVP	6-12 pers.-mois	Branding et workflow unifié immédiat.
VTK/ParaView pour le rendu initial	6-18 pers.-mois	Contrôle viewport très fin.
OCCT plutôt qu'un kernel géométrique	Plusieurs années	Maîtrise totale des algorithmes B-Rep.
Kokkos/PETSc plutôt qu'une abstraction GPU maison	12-24 pers.-mois	Liberté d'API absolue et quelques optimisations spécifiques.
Workers ciblés	3-6 pers.-mois	Un seul binaire universel — qui serait illusoire.
Adaptateurs de processus	Réduit risques ABI/licence	Latence et orchestration supplémentaires.
Profils de formats	Maintenance contenue	Promesse marketing "tout format".

15.3 Ce que l'IA générative peut accélérer

- Scaffolding, wrappers répétitifs, schémas, documentation, tests unitaires et matrices de compatibilité.
- Génération de jeux de tests synthétiques, fuzz harnesses et analyse de logs.
- Migration d'API, exemples Python et documentation multi-plateforme.
- Elle ne remplace pas la validation géométrique, la mesure GPU multi-constructeurs, la sécurité des parseurs, l'audit licence ou les tests sur centres HPC.

15.4 Dépendances externes à obtenir

- Accès à au moins une machine NVIDIA, une AMD ROCm et une CPU ARM64 avant P1.
- Partenariat avec un centre EuroHPC ou universitaire pour profils Slurm et CI jalonnée.
- Contributeurs ou référents OCCT/STEP, DICOM, VTK/ParaView, Kokkos/PETSc et sécurité C++.
- Corpus de conformité dont les licences permettent les tests et, si possible, la redistribution de cas minimaux.

15.5 Effort marginal du profil SALOME

Lot	Ordre de grandeur	Comprend
Spike P0	≈ 0,5 personne-mois	Runtime, capacités, STEP, SMESH/MED, gros artefact, ADR/SBOM.
Connecteur de lecture	1,5 à 2 personnes-mois	MED/XAO/Study, IR, pertes, groupes et champs.
Bidirectionnel	3 à 5 personnes-mois	SHAPER/SMESH/MED, replay, tests et compatibilité 9.16.
YACS/JOBMANAGER/GUI	2 à 4 personnes-mois	Uniquement après cas utilisateur ; maintenance incluse.

Ces charges sont marginales au programme global et ne doivent pas être additionnées mécaniquement lorsque les travaux I/O, conformité et packaging sont déjà comptés ailleurs.

16. Registre de risques et décisions

16.1 Risques prioritaires

Risque	Prob.	Impact	Réduction	Signal d'arrêt
Périmètre trop large	Élevée	Critique	Vertical slice et anti-scope publiés.	Ajout d'un domaine sans cas utilisateur ni mainteneur.
IR devient un nouveau silo	Moyenne	Critique	Adapters + specs indépendantes + export standard.	Impossible d'utiliser les données hors Morphoia.
Kokkos insuffisant	Moyenne	Élevé	Spikes et possibilité de kernels spécialisés.	Écart >2x durable sur kernels clés.
ABI plugins instable	Élevée	Élevé	C ABI minimale, conformance suite, process boundary.	Ruptures à chaque version mineure.
Formats propriétaires	Élevée	Moyen	STEP/connecteurs/SDK optionnels.	Promesse commerciale de fidélité non testée.
Licences incompatibles	Moyenne	Élevé	SBOM, séparation dépôts/processus, revue avant bundle.	Dépendance essentielle non redistribuable.
Pas d'adoption	Moyenne	Critique	Design partners et connecteurs hôtes.	Aucun utilisateur externe actif après P1.
Données médicales	Moyenne	Critique	Recherche-only, de-id, audit, pas de réseau implicite.	Usage clinique envisagé sans processus réglementaire.
Dérive SALOME/MED	Élevée	Élevé	Versionner modules, MED-fichier, formats d'export et images.	Compatibilité interversion non détectée.
Groupes/champs perdus	Moyenne	Critique	Renumérotation explicite, invariants et corpus différentiel.	Conversion silencieuse ou groupes incomplets.
Bridge / gros artefacts	Moyenne	Élevé	CAS/URI, pool chaud, timeouts, limites et reprise.	Payload massif dans contrôle ou overhead >15 %.
Image SALOME contaminée	Moyenne	Élevé	Recette d'abord ; SBOM, sources, patches ; MeshGems exclu.	Binaire non redistribuable dans profil OSS.

16.2 Architecture Decision Records initiaux

ADR	Décision	Statut
ADR-001	Morphoia est une plateforme fédérée, pas un moteur monolithique.	Accepté
ADR-002	C++20 interne, ABI C publique, Python utilisateur.	Accepté
ADR-003	IR JSON canonique + blobs adressés ; pas de format binaire unique.	À valider P0
ADR-004	OCCT/XDE et profil STEP/AP242 comme autorité CAO ouverte.	Accepté
ADR-005	VTK/ITK/PDAL pour les représentations scientifiques.	Accepté
ADR-006	Kokkos workers CPU/CUDA/HIP ; SYCL Tier 2.	À valider P0
ADR-007	DLPack + Arrow C Device comme data plane IA.	À valider P0
ADR-008	Rendu VTK d'abord ; Vulkan/ANARI/Hydra optionnels.	Accepté
ADR-009	HPC via job bundles Slurm/Spark/Apptainer/ADIOS2.	À valider P0
ADR-010	Apache-2.0 OR MIT ; projet gratuit/non commercial par gouvernance, aucune clause NC.	Accepté sous audit
ADR-011	SALOME est un backend CAE optionnel et un oracle de conformité, jamais une dépendance du cœur.	Accepté
ADR-012	MEDCoupling peut être intégré dans un paquet LGPL séparé et lié dynamiquement.	Accepté sous audit
ADR-013	STEP/XAO/MED transportent les artefacts ; CORBA, lorsqu'il intervient, reste interne à SALOME.	Accepté
ADR-014	Le DAG Morphoia et les bundles Slurm restent autoritaires ; YACS/JOBMANAGER sont des hôtes de compatibilité.	Accepté

16.3 Décisions immédiates des 30 prochains jours

- 1 Créer **morphoia-spec** et publier l'anti-scope, les ADR-001 à ADR-014 et le schéma IR minimal.
- 2 Conduire le spike P0-SALOME 9.16 et publier ProbeCapabilities, résultats différentiels, SBOM et décision Go/No-Go.
- 3 Choisir trois datasets minimaux redistribuables : STEP/AP242, DICOM multi-frame/SEG et LAS/LAZ/COPC.
- 4 Implémenter un import de chaque famille avec unités, repères, hash et rapport de perte.
- 5 Écrire un kernel simple en Kokkos et produire les builds CPU, CUDA et HIP avec mesures bout-en-bout.
- 6 Démontrer un aller-retour DLPack vers PyTorch et consigner chaque copie/synchronisation.
- 7 Préparer un job bundle Slurm et demander un Development Access EuroHPC lorsque le worker local est stable.
- 8 Constituer un comité technique léger : géométrie, médical, GPU/HPC, sécurité/licence et utilisateur externe.
- 9 Lancer 20 entretiens de besoin en utilisant le démonstrateur, pas seulement une présentation.

Critère de réussite de v0.1

Un tiers doit pouvoir importer un objet, comprendre les pertes, exécuter une opération sur son matériel, envoyer les données à un framework IA, rejouer le job sur Slurm et vérifier que le résultat provient exactement des mêmes entrées.

A

Annexes techniques

Compatibilités, licences, glossaire et références

Annexe A. Pile logicielle de référence

Domaine	Brique	Licence	Intégration	Rôle
Géométrie exacte	OCCT/OCAF/XDE	LGPL-2.1 + exception	Natif dynamique	STEP/AP242 profil, B-Rep, assemblages
CAE data plane	MEDCoupling/MEDLoader	LGPL-2.1-or-later	Plugin dynamique	Meshes, champs, Gauss, temps, remapping
MED bas niveau	MED-fichier 6.0.1	LGPL-3.0-or-later	Dépendance io-med	Lecture/écriture .med au-dessus de HDF5
CAO SALOME	SHAPER/GEOM	SALOME multilicence ; audit tag	Agent externe	Paramétrique natif, STEP, groupes, XAO
Maillage SALOME	SMESH	LGPL-2.1-or-later + plugins	Agent externe	Hypothèses, groupes, qualité, export MED
Pont géométrique	XAO	Inclus dans modules SALOME	Artefact spécialisé	Forme, topologie, groupes et champs
Workflow compat.	YACS	Audit du tag distribué	Provider P2	Hôte d'un nœud Morphoia job bundle
Jobs compat.	JOBMANAGER	Audit du tag distribué	Provider P2	Ressources et jobs SALOME
Oracle visu	ParaVis	Module SALOME + ParaView	Oracle seulement	ParaView intégré, comparaison de champs
Visualisation	VTK/ParaView	BSD-3-Clause	Natif + plugin	Mesh, volumes, champs, MPI
Imagerie	ITK	Apache-2.0	Natif	Images N-D, segmentation, recalage
DICOM	DCMTK + GDCM	BSD-like / BSD	Module dédié	Objet DICOM + pixel pipeline
Points	PDAL	BSD-3-Clause	Module dédié	LAS/LAZ, COPC/EPT, pipelines
Volumes creux	OpenVDB/NanoVDB	Apache-2.0	Option	Sparse volumes CPU/GPU
Scènes	OpenUSD/MaterialX	TOST-1.0 custom / Apache-2.0	Adapter optionnel	Composition, matériaux, présentation
Diffusion	glTF/GLB	Standard royalty-free	Export	Transmission temps réel
Kernels	Kokkos/Kokkos Kernels	Apache-2.0 + LLVM exception	Worker	CPU/CUDA/HIP/SYCL
Solveurs	PETSc	BSD-2-Clause	Plugin natif	Algèbre, non-linéaire, MPI/GPU
FEM	MFEM/libCEED	BSD-3-Clause	Plugin	Éléments finis accélérés
IA training	PyTorch	BSD-3-Clause	Python externe	Entraînement et recherche
IA inference	ONNX Runtime	MIT	Provider	CPU, CUDA, MIGraphX, OpenVINO...
Tensor exchange	DLPack	Apache-2.0	ABI	Dense tensors et devices
Columnar exchange	Arrow C Data/Device	Apache-2.0	ABI	Attributs et device memory
I/O HPC	ADIOS2	Apache-2.0	Natif	Streaming, checkpoints, BP5
Cluster	Slurm	GPL-2.0+	Client externe	Scheduling
Interconnect	MPI/UCX	Variables	Pile du centre	GPU-aware comms
Packaging HPC	Spack + Apptainer	Apache-2.0 / BSD-3	Outils	Reproductibilité et conteneurs

Cette matrice est une base d'architecture, pas un avis juridique. Les versions, options de compilation, codecs et plugins changent les obligations ; chaque release doit générer une SBOM et conserver les textes de licence correspondants.

Annexe B. Formats et niveaux de service proposés

Format	Lecture	Écriture	Moteur	Réserve
STEP AP242 profil	A/B	A/B	OCCT/XDE	PMI hors profil, topological naming
STEP AP214/AP203	A/B	A/B	OCCT	Variante product data
IGES 5.3	B	B	OCCT	Réparations et perte de sémantique fréquentes
OCCT BREP	A	A	OCCT	Format dépendant du kernel/version
STL	A	A	VTK/OCCT	Aucune unité standard, pas de matériaux
OBJ/MTL	A	A	OCCT/VTK	Variante de conventions
PLY	A	A	VTK/PDAL	Attributs extension-dependent
gITF/GLB	A	A	OCCT/OpenUSD	Dérivé, pas de B-Rep exacte
VTK XML / VTKHDF	A	A	VTK	Profils de champs à fixer
MED	A/B	A/B	MEDCoupling/agent	Version MED, renumérotation, groupes et champs
XAO	B	B	SHAPER/GEOM agent	Pont forme-groupes-champs ; pas un mesh général
SALOME Study .hdf	C	D	Agent SALOME	Opaque, versionné, reproduction seulement
Gmsh / XDMF	B	B	Adapters	Version/attributs spécifiques
LAS/LAZ	A	A	PDAL	CRS et extra dimensions
COPC/EPT	A	B	PDAL	Streaming, services distants
DICOM images	A	B	DCMTK/ITK	Écriture clinique non promise
DICOM SEG/SR/RT	B	B	DCMTK/Slicer	Objets et relations par profil
NIFTI	A	A	ITK	Dérivé ; métadonnées DICOM perdues
OpenVDB	A	A	OpenVDB	3D sparse only
OpenUSD	B	B	OpenUSD	Scène dérivée ; versions de schémas
CATIA/NX/Creo/SW	D	D	Connecteur/SDK	Non open source, dépend de l'outil source

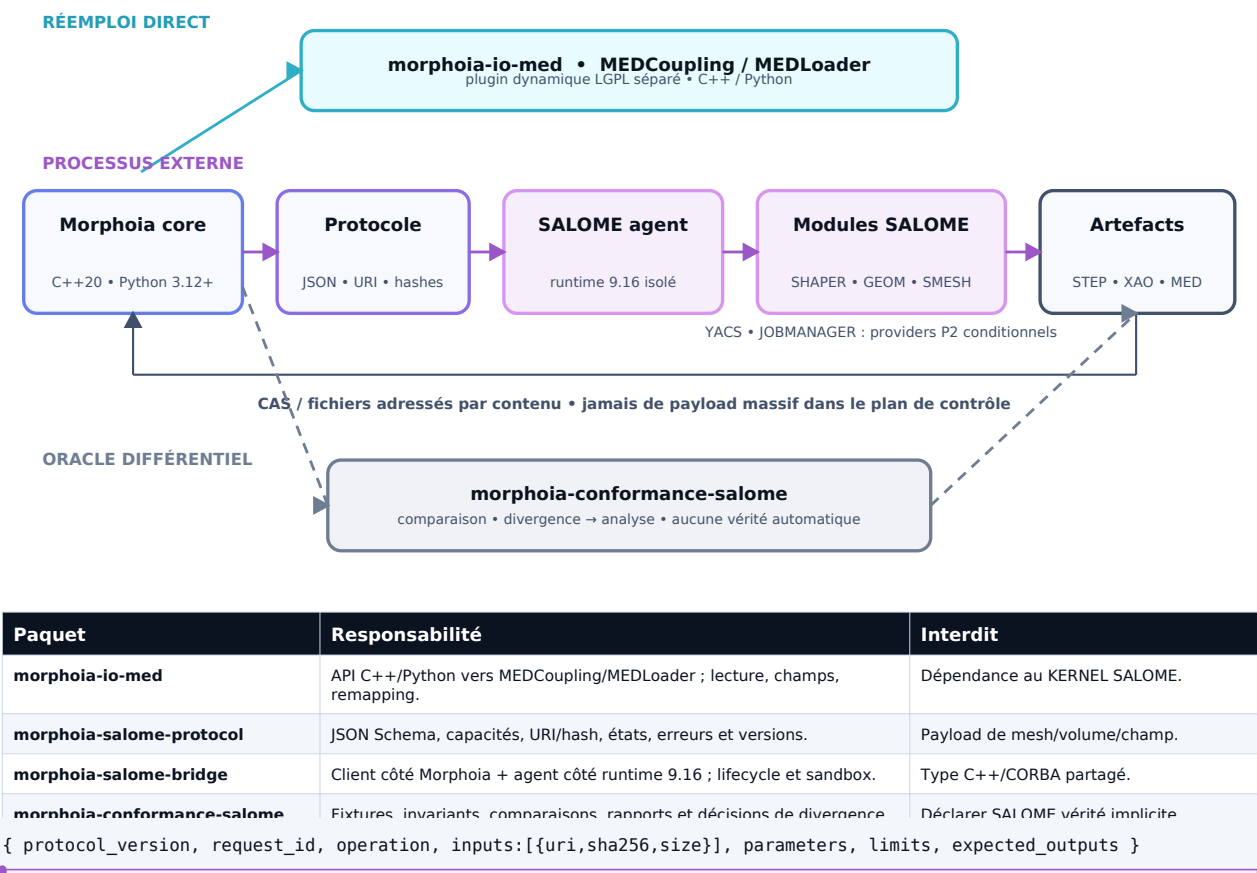
A = round-trip qualifié ; B = fidèle dans un profil ; C = visualisable ; D = expérimental. Les niveaux ci-dessus sont des objectifs de produit et ne deviennent des engagements qu'après validation du corpus de conformité.

Annexe C. Glossaire d'architecture

Terme	Définition
ABI	Contrat binaire entre composants compilés. Une ABI C limite les incompatibilités de compilateurs et de bibliothèques C++.
ANARI	API Khronos haut niveau pour rendre surfaces et volumes via des backends interchangeables.
B-Rep	Représentation exacte par frontières : faces, arêtes, sommets et surfaces paramétriques.
CAS	Content-addressed storage : un contenu immuable est retrouvé par son hash.
DAG	Graphe orienté sans cycle représentant dépendances et provenance d'opérations.
DLPack	Contrat mémoire minimal pour échanger des tenseurs entre runtimes.
IR	Intermediate Representation : enveloppe sémantique reliant données et opérations.
LOD	Level of detail : représentations de précision variable pour interaction et streaming.
MEDCoupling	Bibliothèque C++/Python autonome de maillages, champs, interpolation et transferts.
MED-fichier	API C bas niveau fondée sur HDF5 pour lire et écrire les fichiers .med.
MVX	Famille expérimentale Morphoia d'encodages multi-vues dérivés, évaluée par round-trip et utilité IA.
OCAF/XDE	Frameworks OCCT de documents, assemblages et métadonnées d'échange.
Oracle différentiel	Implémentation de comparaison : une divergence déclenche une analyse, sans présumer la vérité.
ParaVis	Intégration de ParaView dans SALOME, utilisée ici comme oracle de visualisation.
Provider	Implémentation spécifique à un matériel derrière une API commune, notamment ONNX Runtime.
Round-trip	Import puis export/reconstruction avec conservation mesurée de la sémantique attendue.
SHAPER	Modèleur paramétrique SALOME ; STEP/XAO, groupes et historique propre au document.
SMESH	Module SALOME de génération/édition de maillages, hypothèses, groupes et contrôles qualité.
Worker	Binaire headless ciblé sur un backend CPU/GPU et consommant un job bundle.
XAO	Format spécialisé SALOME décrivant forme, topologie, groupes et champs géométriques.
YACS / JOBMANAGER	Hôtes SALOME optionnels de workflow et d'exécution ; jamais autorité du DAG Morphoia.
Zero-copy	Partage du même buffer sans duplication ; dépend du device, du layout, du dtype et de la synchronisation.

Annexe D. Profil technique SALOME 9.16

D.1 Frontière d'exécution et protocole



Paquet	Responsabilité	Interdit
morphaia-io-med	API C++/Python vers MEDCoupling/MEDLoader ; lecture, champs, remapping.	Dépendance au KERNEL SALOME.
morphaia-salome-protocol	JSON Schema, capacités, URI/hash, états, erreurs et versions.	Payload de mesh/volume/champ.
morphaia-salome-bridge	Client côté Morphoia + agent côté runtime 9.16 ; lifecycle et sandbox.	Type C++/CORBA partagé.
morphaia-conformance-salome	Fixtures invariants, comparaisons, rapports et décisions de divergence	Déclarer SALOME vérité implicite

{ protocol_version, request_id, operation, inputs:[{uri,sha256,size}], parameters, limits, expected_outputs }

Cycle de requête

ProbeCapabilities → Submit → Observe/Cancel → Publish. Chaque sortie est immuable, hachée et accompagnée de versions, unités, pertes et logs. L'agent privilégie session-less si la capacité est disponible ; sinon il utilise une session terminal standard confinée.

D.2 Mapping des artefacts et de Morphoia IR

Artefact	Autorité / rôle	Règles Morphoia
STEP / AP242	Frontière CAO générale.	Unités, placements, noms/couleurs et profil AP242 testés ; source conservée.
XAO	Pont SALOME spécialisé de forme, topologie, groupes et champs.	Ne remplace ni STEP général ni MED ; associations et pertes explicites.
MED	Maillages et champs de calcul.	Version logique/effective, supports, composantes, temps et renumérotation enregistrés.
Study .hdf	Artefact opaque de reproduction.	Hash + runtime exact ; jamais IR canonique ni promesse interversion.
Python dump	Replay SHAPER si réellement disponible dans le mode testé.	Hash, provenance et limites ; historique STEP tiers non reconstruit.

Bloc external_refs.salome

Champ	Sémantique
salome_version / modules	Version plateforme, tags/modules effectifs, Python, OS/image et manifest hash.
module / study / study_entry	Référence externe SHAPER/GEOM/SMESH/MEDCoupling ; jamais UUID Morphoia.
mesh_algorithm / hypotheses	Famille, paramètres, sous-maillages, ordre d'application et provenance.
groups / geometry_bindings	Type, membres, filtre, association aux sous-formes et stratégie de sélection.
numbering	Numérotation d'origine, convention de connectivité et table de renumérotation.
field_support	Nœuds, cellules ou points de Gauss ; localisation et interpolation.
components / units / nature	Noms, unités, repères, intensive/extensive, dtype et précision.
time	Pas, itération, ordre, date physique et série multi-étapes.
losses / repairs / dump_hash	Conversions, avertissements, tolérances, métriques et script de replay.

- DICOM et son Frame of Reference restent l'autorité médicale, même si une surface ou un champ est exporté en MED.
- Les IDs SMESH/MED sont des références sources ; toute renumérotation est conservée, jamais assimilée à une identité logique.
- MED 6.0.1 peut produire par défaut un fichier non lisible par des SALOME 9 antérieurs : le profil d'export est explicite.

D.3 Conformité, packaging et exploitation

Famille	Volume minimal	Validation
CAO	20 STEP/AP242	Unités, placements, attributs, topologie, aire/volume.
Maillage	20 SMESH	Types/ordres, connectivité, groupes, sous-maillages et qualité.
Champs	10 MED	Nœuds/cellules/Gauss, unités, composantes, nature et temps.
Grande taille	3 × >2 Gio	URI/hash seulement ; mémoire, reprise et temps mesurés.
Visualisation	ParaView + ParaVis	Structure des champs identique ou divergence expliquée.
Isolation	Analyse binaire + import	Aucun SALOME/Qt/PyQt/omniORB dans morphoia-core.
Performance	Pool froid et chaud	Overhead chaud <15 % sur opération longue ; startup séparé.

Profils de déploiement

Profil	Contenu	Décision
OSS de référence	Core + protocole + io-med optionnel ; aucun MeshGems.	CI et distribution officielles.
Agent local	Recette reproductible ; installation SALOME 9.16 fournie par l'utilisateur.	Chemin P0-P1 préféré.
Image SALOME auditée	Runtime distinct, SBOM, licences/notices, sources exactes et patches.	Après audit, jamais implicite.
GUI SALOME	Module facultatif pouvant engager PyQt/PyQt-Charts GPLv3.	P2, distribution séparée.
Site HPC	Slurm direct autoritaire ; YACS/JOBMANAGER seulement si utiles.	Profil local qualifié.

Gate P0

Aucun No-Go n'est contourné : dépendance du cœur, perte silencieuse d'unité/dimension, groupes incomplets, MeshGems dans l'OSS ou payload massif dans le contrôle. Une divergence de l'oracle ouvre une analyse documentée.

Annexe E. Références officielles

Références consultées le 6 août 2026. Les liens sont cliquables. Les versions exactes devront être verrouillées au lancement de P0.

- [R01] SALOME Platform 9.16 — annonce officielle du 27 juillet 2026
<https://www.salome-platform.org/?p=61308>
- [R02] Alliance for OpenUSD — Core Specification 1.0
<https://aousd.org/news/core-spec-announcement/>
- [R03] VTK — présentation, parallélisme et licence
<https://docs.vtk.org/en/latest/about.html>
- [R04] ITK — documentation officielle
<https://docs.itk.org/en/latest/>
- [R05] PDAL — Point Data Abstraction Library
<https://pdal.io/>
- [R06] Open CASCADE — interface STEP
https://dev.opencascade.org/doc/overview/html/occt_user_guides__step.html
- [R07] Open CASCADE — Extended Data Exchange (XDE)
https://dev.opencascade.org/doc/overview/html/occt_user_guides__xde.html
- [R08] 3D Slicer — image computing platform
<https://slicer.org/>
- [R09] SOFA — fonctionnalités et architecture
<https://www.sofa-framework.org/about/features/>
- [R10] FreeCAD — présentation officielle
https://wiki.freecad.org/About_FreeCAD
- [R11] interTwin — Digital Twin Engine open source
<https://www.intertwin.eu/ker-1-interdisciplinary-digital-twin-engine>
- [R12] Commission européenne — European Virtual Human Twins
<https://digital-strategy.ec.europa.eu/en/policies/virtual-human-twins>
- [R13] EDITH — European Virtual Human Twin vision
<https://www.edith-csa.eu/>
- [R14] Kokkos — configuration des backends
<https://kokkos.org/kokkos-core-wiki/get-started/configuration-guide.html>
- [R15] Khronos — Vulkan
<https://www.khronos.org/vulkan/>
- [R16] Khronos — ANARI
<https://www.khronos.org/anari/>
- [R17] DLPack — spécification Python
https://dmlc.github.io/dlpack/latest/python_spec.html
- [R18] Apache Arrow — C Device Data Interface
<https://arrow.apache.org/docs/format/CDeviceDataInterface.html>
- [R19] ONNX Runtime — Execution Providers
<https://onnxruntime.ai/docs/execution-providers/>
- [R20] NVIDIA — licence cuDNN
<https://docs.nvidia.com/deeplearning/cudnn/backend/latest/reference/eula.html>
- [R21] AMD — licences des composants ROCm
<https://rocm.docs.amd.com/en/latest/about/license.html>
- [R22] PETSc — configuration GPU et Kokkos
<https://petsc.org/release/install/install/>
- [R23] MFEM — backends de calcul
https://docs.mfem.org/4.8/device_8cpp_source.html
- [R24] Slurm — overview
<https://slurm.schedmd.com/overview.html>
- [R25] Apptainer — support GPU
<https://apptainer.org/docs/user/main/gpu.html>
- [R26] Spack — environnements reproductibles
<https://spack.readthedocs.io/en/latest/environments.html>
- [R27] ADIOS2 — framework I/O haute performance
<https://adios2.readthedocs.io/en/latest/>
- [R28] OpenUCX — FAQ GPU CUDA et ROCm
<https://openucx.readthedocs.io/en/master/faq.html>
- [R29] EuroHPC — Access Calls Technical Guidelines
https://www.eurohpc-ju.europa.eu/document/download/7ac29264-106d-497c-9683-42aec21bf10f_en?filename=Technical_Guidelines-all+calls-26.05.2025.pdf
- [R30] EuroHPC — Development Access
https://www.eurohpc-ju.europa.eu/eurohpc-ju-call-proposals-development-access_en
- [R31] OpenUSD — documentation
<https://openusd.org/>
- [R32] Khronos — glTF
<https://www.khronos.org/glTF/>
- [R33] OpenVDB — présentation et licence
<https://www.openvdb.org/>
- [R34] Godot — GDEXTension et pont C
https://docs.godotengine.org/en/4.4/getting_started/step_by_step/scripting_languages.html
- [R35] Blender — Python API
https://docs.blender.org/api/current/info_overview.html
- [R36] ParaView — plugins et filtres programmables
<https://docs.paraview.org/en/v5.11.2/ReferenceManual/pythonProgrammableFilter.html>
- [R37] OpenUSD — plugins tiers et Hydra
<https://openusd.org/24.08/plugins.html>

- [R38] PyTorch — gouvernance et licence BSD
<https://docs.pytorch.org/docs/stable/community/governance.html>
- [R39] Open CASCADE — licence LGPL-2.1 avec exception
<https://dev.opencascade.org/resources/licensing>
- [R40] VTK — licence BSD-3-Clause
<https://docs.vtk.org/en/latest/about.html#license>
- [R41] Apache Software Foundation — Apache License 2.0
<https://www.apache.org/licenses/LICENSE-2.0>
- [R42] Open Source Initiative — Open Source Definition
<https://opensource.org/osd>
- [R43] REUSE — conformité et SPDX
<https://reuse.software/>
- [R44] DICOM Standard — édition courante et DICOMweb
<https://www.dicomstandard.org/current/>
- [R45] EuroHPC — inventaire des supercalculateurs
https://www.eurohpc-ju.europa.eu/supercomputers/our-supercomputers_en
- [R46] Kokkos — licence Apache-2.0 avec exception LLVM
<https://github.com/kokkos/kokkos/blob/develop/LICENSE>
- [R47] OpenUSD — licence TOST-1.0 et notices tierces
<https://github.com/PixarAnimationStudios/OpenUSD/blob/dev/LICENSE.txt>
- [R48] SALOME 9.16.0 — notes de version officielles (juin 2026)
https://www.salome-platform.org/wp-content/uploads/2026/07/SALOME_9_16_0_Release_Notes.pdf
- [R49] NVIDIA Omniverse — accord de licence
https://docs.omniverse.nvidia.com/dev-guide/latest/common/NVIDIA_Omniverse_License_Agreement.html
- [R50] NVIDIA Omniverse — conditions d'utilisation et accès 2026
https://docs.omniverse.nvidia.com/ov/latest/omniverse_quickstart-guide.html
- [R51] MFEM — fonctionnalités et backends
<https://mfem.org/features/>
- [R52] FEniCSx — projet open source de calcul par éléments finis
<https://fenicsproject.org/>
- [R53] SimVascular — plateforme open source de simulation cardiovasculaire
<https://simvascular.github.io/>
- [R54] FEBio — suite de simulation biomécanique
<https://febio.org/>
- [R55] SALOME 9.16 — architecture KERNEL/CORBA
https://docs.salome-platform.org/latest/tui/GUI/salome_architecture_page.html
- [R56] MEDCoupling 9.16 — guide développeur C++/Python
<https://docs.salome-platform.org/latest/dev/MEDCoupling/developer/index.html>
- [R57] SMESH — dépôt officiel, prérequis et licence
<https://github.com/SalomePlatform/smesh>
- [R58] MEDCoupling — constellation MED, MEDLoader et MED-fichier
<https://docs.salome-platform.org/latest/dev/MEDCoupling/developer/library.html>
- [R59] SHAPER 9.16 — introduction, paramétrique et formats
<https://docs.salome-platform.org/latest/gui/SHAPER/General/Introduction.html>
- [R60] MEDLoader 9.16 — lecture et écriture des fichiers MED
<https://docs.salome-platform.org/latest/dev/MEDCoupling/developer/medloader.html>
- [R61] ParaVis 9.16 — architecture de l'intégration ParaView
<https://docs.salome-platform.org/latest/dev/PARAVIS/index.html>
- [R62] SALOME — page officielle des licences et prérequis
https://www.salome-platform.org/?page_id=2622
- [R63] Riverbank — PyQt, GPLv3 ou licence commerciale
<https://riverbankcomputing.com/software/pyqt/intro>
- [R64] Riverbank — PyQt-Charts, GPLv3 ou licence commerciale
<https://riverbankcomputing.com/software/pyqtchart/intro>
- [R65] SHAPER 9.16 — Python dump et limites de la commande TUI
<https://docs.salome-platform.org/latest/gui/SHAPER/ExchangePlugin/dumpFeature.html>
- [R66] YACS 9.16 — principes des graphes de calcul
<https://docs.salome-platform.org/latest/gui/YACS/principes.html>
- [R67] JOBMANAGER — concept de job et ressources
<https://docs.salome-platform.org/latest/gui/JOBMANAGER/job.html>
- [R68] GEOM — import/export XAO
https://docs.salome-platform.org/9/gui/GEOM/import_export_geom_obj_page.html
- [R69] SALOME KERNEL 9.16 — modes terminal et session-less
https://docs.salome-platform.org/latest/tui/KERNEL/running_salome_page.html
- [R70] MEDCoupling — dépôt officiel et licence LGPL-2.1-or-later
<https://github.com/SalomePlatform/medcoupling>

Conclusion

Morphoia Engine est justifié s'il reste une couche de continuité mesurable et ouverte. Sa réussite se jugera moins au nombre de formats affichés qu'à la possibilité de conserver l'intention, expliquer les pertes, réutiliser les meilleurs moteurs existants et rejouer le même calcul sur une infrastructure différente.